



Разработчик gRPC-сервисов в Python и Go

Модуль 1 - Введение в gRPC (Junior-ready)

Урок 1 - gRPC vs REST vs GraphQL

Урок 2 - Протокол HTTP2 и бинарная сериализация Protobuf

Урок 3 - Первый gRPC-сервис HelloWorld на Go

Урок 4 - Практика простой калькулятор через gRPC на Python

Урок 5 - Викторина 1

Шаг 01:

Миссия 1

В вашей команде новичок путается в ключевых понятиях gRPC. Помогите сопоставить термины с их описаниями.

Подсказка: Вспомните, что отвечает за описание интерфейса, что за транспорт, а что за реализацию.

Термины:

Ответ:

- Protocol Buffers
- Stub
- Service
- Channel

Определения:

1. Объект, представляющий подключение к gRPC-серверу
 2. Формат описания сообщений и сервисов
 3. Автоматически сгенерированный клиентский код для вызова методов сервиса
 4. Определение набора методов, которые реализует сервер
- **Protocol Buffers** — Формат описания сообщений и сервисов
 - **Stub** — Автоматически сгенерированный клиентский код для вызова методов сервиса
 - **Service** — Определение набора методов, которые реализует сервер
 - **Channel** — Объект, представляющий подключение к gRPC-серверу

🔮 Шаг 02:

Миссия 2

Команда обсуждает режимы взаимодействия в gRPC. Помогите сопоставить тип вызова с его описанием.

Подсказка: Подумайте, как данные передаются и кто инициирует поток.

Типы вызовов:

Ответ:

- Unary RPC
- Server streaming
- Client streaming
- Bidirectional streaming

Описание:

1. Клиент отправляет один запрос и получает один ответ
 2. Клиент отправляет один запрос, сервер отправляет поток ответов
 3. Клиент отправляет поток запросов, сервер отвечает одним сообщением
 4. **Оба** — клиент и сервер — обмениваются потоками сообщений
- **Unary RPC** — Клиент отправляет один запрос и получает один ответ
 - **Server streaming** — Клиент отправляет один запрос, сервер отправляет поток ответов
 - **Client streaming** — Клиент отправляет поток запросов, сервер отвечает одним сообщением
 - **Bidirectional streaming** — Оба — клиент и сервер — обмениваются потоками сообщений

🔮 Шаг 03:

Миссия 3

Ваша команда настраивает аутентификацию и хочет понять, какие механизмы можно использовать с gRPC. Сопоставьте типы аутентификации с описаниями.

Подсказка: Подумайте, какие методы подходят для безопасных соединений и передачи токенов.

Типы аутентификации:

Ответ:

- SSL/TLS
- OAuth 2.0
- API Key
- JWT (JSON Web Token)

Описание:

1. Шифрование канала с проверкой сертификатов
 2. Токен для доступа с правами и сроком действия
 3. Простой ключ доступа в заголовке запроса
 4. Протокол авторизации с выдачей токенов доступа
- **SSL/TLS** — Шифрование канала с проверкой сертификатов
 - **OAuth 2.0** — Протокол авторизации с выдачей токенов доступа
 - **API Key** — Простой ключ доступа в заголовке запроса
 - **JWT (JSON Web Token)** — Токен для доступа с правами и сроком действия

Шаг 04:

Миссия 4

Задача: Выстраиваем процесс обработки запроса в gRPC-сервисе. Расположите шаги в правильном порядке:

1. Отправка ответа клиенту
2. Получение запроса от клиента
3. Валидация и обработка данных
4. Установка соединения с клиентом

Подсказка: Помните, что перед обработкой нужно установить соединение и получить данные.

Ответ:

- Установка соединения с клиентом
- Получение запроса от клиента
- Валидация и обработка данных
- Отправка ответа клиенту

Шаг 05:

Миссия 5

Задача: При оптимизации gRPC-сервиса необходимо правильно расставить шаги для уменьшения задержек:

1. Параллелизация потоков данных
2. Минимизация размера сообщений
3. Использование сжатия
4. Настройка таймаутов и повторных попыток

Подсказка: Сначала уменьшите данные, потом оптимизируйте их передачу, и только потом управляйте поведением сети.

Ответ:

- Минимизация размера сообщений
- Использование сжатия
- Параллелизация потоков данных
- Настройка таймаутов и повторных попыток

Шаг 06:

Миссия 6

Задача: При работе с protobuf необходимо придерживаться правильного порядка шагов. Расположите их последовательно:

1. Определение структуры сообщений и сервисов в .proto-файле
2. Генерация кода на выбранном языке
3. Реализация бизнес-логики в сервисе
4. Запуск сервера и клиентских приложений

Подсказка: Сначала описываем контракт, потом генерируем код, пишем логику и запускаем.

Ответ:

- Определение структуры сообщений и сервисов в .proto-файле
- Генерация кода на выбранном языке
- Реализация бизнес-логики в сервисе
- Запуск сервера и клиентских приложений

Шаг 07:

Миссия 7

Задача: Соберите правильный порядок шагов для отладки gRPC-приложения при возникновении ошибок соединения:

1. Проверка конфигурации SSL/TLS
2. Использование инструментов типа grpcurl для тестирования вызовов
3. Анализ логов сервера и клиента
4. Проверка доступности сети и портов

Подсказка: Начинайте с базовой сетевой диагностики, потом проверяйте безопасность и тестируйте вызовы.

Ответ:

- Проверка доступности сети и портов
- Проверка конфигурации SSL/TLS
- Использование инструментов типа grpcurl для тестирования вызовов
- Анализ логов сервера и клиента

Урок 6 - Викторина 2

Шаг 01:

Миссия 1

Вам нужно сопоставить понятия из мира gRPC с их правильными описаниями.

Понятия:

Ответ:

- Unary RPC
- Client Streaming
- Server Streaming
- Bidirectional Streaming

Определения:

1. Один запрос – один ответ.
2. Клиент отправляет поток сообщений, сервер один ответ.
3. Клиент отправляет один запрос, сервер отвечает потоком сообщений.

4. Обе стороны обмениваются потоками сообщений одновременно.

Подсказка: Подумайте, как различаются типы взаимодействия в gRPC по количеству сообщений.

- **Unary RPC** — Один запрос – один ответ.
- **Client Streaming** — Клиент отправляет поток сообщений, сервер один ответ.
- **Server Streaming** — Клиент отправляет один запрос, сервер отвечает потоком сообщений.
- **Bidirectional Streaming** — Обе стороны обмениваются потоками сообщений одновременно.

Шаг 02:

Миссия 2

Сопоставьте основные компоненты gRPC с их функциями:

Компоненты:

Ответ:

- Stub
- Channel
- Interceptor
- Service Definition

Функции:

1. Определяет методы и сообщения сервиса
2. Клиентская прокси для вызова удалённых методов
3. Средство перехвата и обработки вызовов
4. Устанавливает соединение с сервером

Подсказка: Подумайте, кто что делает при вызове gRPC сервиса.

- **Service Definition** — Определяет методы и сообщения сервиса
- **Stub** — Клиентская прокси для вызова удалённых методов
- **Interceptor** — Средство перехвата и обработки вызовов
- **Channel** — Устанавливает соединение с сервером

Шаг 03:

Миссия 3

Команда обсуждает типы ошибок в gRPC. Сопоставьте тип ошибки с её причиной:

Типы ошибок:

Ответ:

- Deadline Exceeded
- Unavailable
- Internal
- Invalid Argument

Причины:

1. Сервер не отвечает вовремя
2. Неправильные данные в запросе
3. Временная недоступность сервиса
4. Внутренняя ошибка сервера

Подсказка: Обратите внимание, что gRPC возвращает конкретные коды ошибок для разных ситуаций.

- **Deadline Exceeded** — Сервер не отвечает вовремя
- **Invalid Argument** — Неправильные данные в запросе
- **Unavailable** — Временная недоступность сервиса
- **Internal** — Внутренняя ошибка сервера

🔮 Шаг 04:

Миссия 4

Задача: Расположите шаги создания простого gRPC сервиса в правильном порядке:

1. Написание .proto файла с определением сервиса и сообщений
2. Генерация кода из .proto с помощью protoc
3. Реализация серверной логики на выбранном языке
4. Запуск сервера и клиента для тестирования

Подсказка: Процесс начинается с определения интерфейса, затем подготовка кода, реализация и тестирование.

Ответ:

- Написание .proto файла с определением сервиса и сообщений
- Генерация кода из .proto с помощью protoc
- Реализация серверной логики на выбранном языке
- Запуск сервера и клиента для тестирования

🔮 Шаг 05:

Миссия 5

Задача: Упорядочите этапы отладки gRPC сервиса при возникновении проблемы с подключением:

1. Проверка правильности адреса и порта сервера
2. Анализ логов сервера и клиента
3. Проверка сетевых настроек и доступности сервера
4. Запуск тестового клиента с простым запросом

Подсказка: Начинайте с базовых проверок, затем углубляйтесь в детали.

Ответ:

- Проверка правильности адреса и порта сервера
- Проверка сетевых настроек и доступности сервера
- Запуск тестового клиента с простым запросом
- Анализ логов сервера и клиента

🔮 Шаг 06:

Миссия 6

Задача: Расположите этапы оптимизации gRPC приложения по снижению задержек:

1. Использование сжатия сообщений
2. Профилирование и анализ узких мест
3. Настройка таймаутов и дедлайнов
4. Реализация кэширования на стороне клиента

Подсказка: Начинайте с измерения, затем применяйте улучшения.

Ответ:

- Профилирование и анализ узких мест
- Использование сжатия сообщений
- Настройка таймаутов и дедлайнов
- Реализация кэширования на стороне клиента

 **Шаг 07:**

Миссия 7

Задача: Упорядочьте шаги взаимодействия клиента и сервера в gRPC при Unary вызове:

1. Клиент формирует и отправляет запрос
2. Сервер принимает запрос и обрабатывает его
3. Сервер отправляет ответ клиенту
4. Клиент получает ответ и завершает вызов

Подсказка: Внимательно проследите, как происходит обмен сообщениями при обычном RPC.

Ответ:

- Клиент формирует и отправляет запрос
- Сервер принимает запрос и обрабатывает его
- Сервер отправляет ответ клиенту
- Клиент получает ответ и завершает вызов

Модуль 2 - Основы gRPC-программирования

 **Урок 1 - Определение сервисов в .proto**

 **Урок 2 - Unary RPC + ошибки/статусы**

 **Урок 3 - Генерация кода на Go, Python, PHP**

 **Урок 4 - gRPC и типы данных**

 **Урок 5 -  Практика сервис переводчика (строка → другая строка)**

 **Урок 6 - Викторина 1**

 **Шаг 01:**

Миссия 1

Разработчик Анна запускает своё первое gRPC-приложение. Какой протокол используется для передачи данных между клиентом и сервером?

Подсказка: gRPC основан на современном высокопроизводительном протоколе от Google.

Ответ:

- **HTTP/2**
- HTTP/1.1
- FTP
- WebSocket

Шаг 02:

Миссия 2

Команда решает, что нужно использовать Protocol Buffers (protobuf) для определения сообщений в gRPC. Почему protobuf так популярен в gRPC?

Подсказка: Обратите внимание на формат данных и производительность.

Ответ:

- Он обеспечивает компактное и быстрое сериализованное представление данных
- Это формат JSON, удобный для людей
- Позволяет хранить данные в SQL-базе
- Это только язык программирования для серверов

Шаг 03:

Миссия 3

Владимир внедряет Unary RPC вызов в сервисе. Что характерно для Unary RPC?

Подсказка: Подумайте о количестве сообщений от клиента и сервера.

Ответ:

- Клиент отправляет один запрос, сервер возвращает один ответ
- Клиент и сервер обмениваются потоками сообщений одновременно
- Сервер только отправляет сообщения, клиент только принимает
- Клиент отправляет несколько запросов подряд без ответа

Шаг 04:

Миссия 4

Команда обсуждает разницу между синхронным и асинхронным вызовом в gRPC. Что из перечисленного — преимущество асинхронных вызовов?

Подсказка: Подумайте о том, как клиент может продолжать работу.

Ответ:

- Клиент не блокируется и может выполнять другие задачи во время ожидания ответа
- Запросы всегда обрабатываются быстрее
- Клиент получает ответ мгновенно
- Асинхронный вызов требует меньше памяти

Шаг 05:

Миссия 5

Задача: Сопоставьте типы gRPC-вызовов с их описаниями.

Типы:

Ответ:

- Unary RPC
- Server Streaming RPC
- Client Streaming RPC
- Bidirectional Streaming RPC

Описание:

1. Один запрос от клиента и один ответ от сервера

2. Сервер отправляет поток сообщений в ответ на один запрос клиента
3. Клиент отправляет поток сообщений, сервер отвечает одним сообщением
4. Оба обмениваются потоками сообщений одновременно

Подсказка: Вспомните, кто инициирует и сколько сообщений обменивается.

- **Unary RPC** — Один запрос от клиента и один ответ от сервера
- **Server Streaming RPC** — Сервер отправляет поток сообщений в ответ на один запрос клиента
- **Client Streaming RPC** — Клиент отправляет поток сообщений, сервер отвечает одним сообщением
- **Bidirectional Streaming RPC** — Оба обмениваются потоками сообщений одновременно

Шаг 06:

Миссия 6

Задача: Сопоставьте основные компоненты gRPC и их роли.

Компоненты:

Ответ:

- Protocol Buffers
- Stub
- Channel
- Server

Роли:

1. Сериализация и десериализация сообщений
2. Объект, через который клиент вызывает удалённые методы
3. Абстракция для управления соединением между клиентом и сервером
4. Обрабатывает запросы клиентов и отправляет ответы

Подсказка: Каждый компонент играет свою роль в коммуникации и обработке данных.

- **Protocol Buffers** — Сериализация и десериализация сообщений
- **Stub** — Объект, через который клиент вызывает удалённые методы
- **Channel** — Абстракция для управления соединением между клиентом и сервером
- **Server** — Обрабатывает запросы клиентов и отправляет ответы

Шаг 07:

Миссия 7

Задача: Сопоставьте этапы разработки gRPC-сервиса с их описаниями.

Этапы:

Ответ:

- Определение .proto файла
- Генерация кода
- Реализация сервера
- Написание клиента

Описание:

1. Создание файла с описанием сервисов и сообщений
2. Автоматическое создание клиентских и серверных классов

3. Создание логики обработки запросов
4. Использование сгенерированного кода для вызова RPC

Подсказка: Следуйте логике от спецификации к работе.

- **Определение .proto файла** — Создание файла с описанием сервисов и сообщений
- **Генерация кода** — Автоматическое создание клиентских и серверных классов
- **Реализация сервера** — Создание логики обработки запросов
- **Написание клиента** — Использование сгенерированного кода для вызова RPC

Шаг 08:

Миссия 8

Вопрос: Упорядочьте этапы настройки gRPC-сервиса от начала до готовности к работе.

1. Реализация логики сервера
2. Определение сервиса в .proto файле
3. Генерация серверного и клиентского кода
4. Запуск сервера и проверка соединения

Подсказка: Начинайте с описания, потом генерируйте код, реализуйте логику и запускайте.

Ответ:

- Определение сервиса в .proto файле
- Генерация серверного и клиентского кода
- Реализация логики сервера
- Запуск сервера и проверка соединения

Шаг 09:

Миссия 9

Вопрос: Расположите шаги для обработки потока сообщений от клиента в Client Streaming RPC.

1. Клиент отправляет несколько сообщений в поток
2. Сервер обрабатывает сообщения по мере поступления
3. Клиент завершает отправку сообщений
4. Сервер отправляет один итоговый ответ

Подсказка: Внимательно следите за последовательностью потокового взаимодействия.

Ответ:

- Клиент отправляет несколько сообщений в поток
- Сервер обрабатывает сообщения по мере поступления
- Клиент завершает отправку сообщений
- Сервер отправляет один итоговый ответ

Шаг 10:

Миссия 10

Вопрос: Упорядочьте шаги для отладки и тестирования gRPC-сервиса после его написания.

1. Написание тестов для RPC методов
2. Запуск сервера
3. Использование клиента для вызова методов
4. Анализ результатов и исправление ошибок

Подсказка: Помните, что тестирование начинается с подготовки, а заканчивается исправлением.

Ответ:

- Запуск сервера
- Написание тестов для RPC методов
- Использование клиента для вызова методов
- Анализ результатов и исправление ошибок

Урок 7 - Викторина 2

Шаг 01:

Миссия 1

Какой протокол используется в основе gRPC для передачи данных?

Подсказка: Это протокол HTTP следующего поколения.

Ответ:

- **HTTP/2**
- HTTP/1.1
- WebSocket
- FTP

Шаг 02:

Миссия 2

Что из перечисленного — формат сериализации сообщений в gRPC?

Подсказка: Это формат от Google, с компактной структурой.

Ответ:

- **Protocol Buffers (Protobuf)**
- JSON
- XML

Шаг 03:

Миссия 3

Какая из моделей вызова поддерживается в gRPC?

Подсказка: Подумайте о потоках данных между клиентом и сервером.

Ответ:

- **Unary (одинарный вызов)**
- **Server streaming**
- **Client streaming**
- **Bidirectional streaming**

Шаг 04:

Миссия 4

Сопоставьте понятия gRPC с их описаниями:

Ответ:

- Unary RPC

- Server streaming RPC
- Client streaming RPC
- Bidirectional streaming RPC

1. **Один запрос** — один ответ
2. **Один запрос** — несколько ответов
3. **Несколько запросов** — один ответ
4. Двухнаправленный обмен сообщениями

Подсказка: Обратите внимание на направление и количество сообщений.

- **Unary RPC** — Один запрос — один ответ
- **Server streaming RPC** — Один запрос — несколько ответов
- **Client streaming RPC** — Несколько запросов — один ответ
- **Bidirectional streaming RPC** — Двухнаправленный обмен сообщениями

Шаг 05:

Миссия 5

Сопоставьте элементы gRPC с их ролями:

Ответ:

- Stub
 - Service definition
 - Protocol Buffers
 - Channel
1. Автоматически сгенерированный клиентский код
 2. Описание методов и сообщений
 3. Формат сериализации данных
 4. Установленное соединение для обмена

Подсказка: Вспомните этапы коммуникации клиента и сервера.

- **Stub** — Автоматически сгенерированный клиентский код
- **Service definition** — Описание методов и сообщений
- **Protocol Buffers** — Формат сериализации данных
- **Channel** — Установленное соединение для обмена

Шаг 06:

Миссия 6

Сопоставьте этапы создания gRPC-сервиса:

Ответ:

- Определение сервиса в proto
 - Генерация кода
 - Реализация сервера
 - Запуск сервера
1. Первый шаг
 2. Второй шаг
 3. Третий шаг
 4. Четвертый шаг

Подсказка: Подумайте о порядке действий от описания до запуска.

- **Определение сервиса в proto** — Первый шаг
- **Генерация кода** — Второй шаг
- **Реализация сервера** — Третий шаг
- **Запуск сервера** — Четвертый шаг

Шаг 07:

Миссия 7

Расположите шаги для настройки gRPC-клиента в правильном порядке:

1. Создание канала связи
2. Создание Stub (прокси клиента)
3. Вызов удаленного метода
4. Обработка ответа

Подсказка: Подумайте, как происходит подготовка и выполнение вызова.

Ответ:

- Создание канала связи
- Создание Stub (прокси клиента)
- Вызов удаленного метода
- Обработка ответа

Шаг 08:

Миссия 8

Упорядочьте этапы обработки запроса на стороне сервера gRPC:

1. Получение запроса
2. Обработка логики
3. Формирование ответа
4. Отправка ответа клиенту

Подсказка: Рассмотрите полный цикл обработки.

Ответ:

- Получение запроса
- Обработка логики
- Формирование ответа
- Отправка ответа клиенту

Шаг 09:

Миссия 9

Расположите этапы от генерации Protobuf сообщений до выполнения RPC:

1. Определение сообщений и сервисов в .proto
2. Генерация кода из .proto
3. Компиляция и сборка приложения
4. Выполнение вызова RPC

Подсказка: Проследите путь данных от описания до исполнения.

Ответ:

- Определение сообщений и сервисов в .proto
- Генерация кода из .proto
- Компиляция и сборка приложения
- Выполнение вызова RPC

Модуль 3 - gRPC Streaming

Урок 1 - Server-side streaming

Урок 2 - Client-side streaming

Урок 3 - Bidirectional streaming

Урок 4 - Сравнение стриминга и WebSocket

Урок 5 - Практика чат с двусторонним стримингом

Урок 6 - Викторина 1

Шаг 01:

Миссия 1

Команда начала использовать потоковую передачу в gRPC, но столкнулась с вопросом: какой вид стриминга позволяет клиенту отправлять несколько сообщений, а сервер — ответить одним?

Подсказка: Подумайте, кто инициирует поток сообщений и кто ждет единственный ответ.

Ответ:

- **Client streaming**
- Unary
- Server streaming
- Bidirectional streaming

Шаг 02:

Миссия 2

Разработчики задумались, как gRPC обеспечивает поддержку одновременной отправки и получения сообщений между клиентом и сервером. Как называется такой тип стриминга?

Подсказка: Это двунаправленный обмен сообщениями в реальном времени.

Ответ:

- **Bidirectional streaming**
- Unary
- Server streaming

Шаг 03:

Миссия 3

Команда обсуждает, какой формат сериализации используется в gRPC, чтобы обмениваться компактными и быстрыми сообщениями.

Подсказка: Это бинарный формат от Google, широко применяемый с gRPC.

Ответ:

- **Protocol Buffers**
- JSON
- XML

Шаг 04:

Миссия 4

Команда проводит обучение и просит сопоставить типы gRPC стриминга с их описаниями.

Ответ:

- Unary
 - Server streaming
 - Client streaming
 - Bidirectional streaming
1. **Один запрос** — один ответ
 2. **Один запрос** — много ответов
 3. **Много запросов** — один ответ
 4. **Много запросов** — много ответов

Подсказка: Вспомните, кто инициирует и сколько сообщений отправляется в каждом направлении.

- **Unary** — Один запрос — один ответ
- **Server streaming** — Один запрос — много ответов
- **Client streaming** — Много запросов — один ответ
- **Bidirectional streaming** — Много запросов — много ответов

Шаг 05:

Миссия 5

Для успешной работы с gRPC-стримингом команда хочет сопоставить ключевые компоненты с их назначением.

Ответ:

- Stub
 - Channel
 - Service definition
 - Protocol Buffers
1. Клиентский прокси для вызовов
 2. Связь между клиентом и сервером
 3. Определение методов и сообщений
 4. Формат сериализации данных

Подсказка: Обратите внимание на роли каждого элемента в процессе обмена данными.

- **Stub** — Клиентский прокси для вызовов
- **Channel** — Связь между клиентом и сервером
- **Service definition** — Определение методов и сообщений
- **Protocol Buffers** — Формат сериализации данных

Шаг 06:

Миссия 6

Команда хочет понять порядок действий при организации потокового RPC. Помогите сопоставить этапы с их описаниями.

Ответ:

- Определение сервиса в proto-файле
- Генерация кода на основе proto
- Реализация логики сервера
- Запуск сервера

1. Первый этап
2. Второй этап
3. Третий этап
4. Четвертый этап

Подсказка: Вспомните, как начинается работа с gRPC и что происходит дальше.

- **Определение сервиса в proto-файле** — Первый этап
- **Генерация кода на основе proto** — Второй этап
- **Реализация логики сервера** — Третий этап
- **Запуск сервера** — Четвертый этап

Шаг 07:

Миссия 7

Расположите шаги для настройки потокового gRPC-клиента в правильном порядке:

1. Создание канала связи
2. Создание Stub (прокси клиента)
3. Вызов метода с использованием стриминга
4. Обработка потоковых ответов

Подсказка: Подумайте, как строится коммуникация от соединения до получения данных.

Ответ:

- Создание канала связи
- Создание Stub (прокси клиента)
- Вызов метода с использованием стриминга
- Обработка потоковых ответов

Шаг 08:

Миссия 8

Упорядочьте этапы обработки потокового запроса на сервере:

1. Получение потоковых сообщений от клиента
2. Обработка данных в логике сервера
3. Отправка ответных сообщений клиенту
4. Завершение стрима

Подсказка: Рассмотрите полный жизненный цикл обработки стрима на сервере.

Ответ:

- Получение потоковых сообщений от клиента
- Обработка данных в логике сервера
- Отправка ответных сообщений клиенту
- Завершение стрима

 Шаг 09:

Миссия 9

Расположите этапы создания и тестирования потокового gRPC-сервиса:

1. Определение сообщений и сервисов в .proto
2. Генерация кода
3. Компиляция и запуск приложения
4. Тестирование потокового взаимодействия

Подсказка: Проследите путь от определения API до проверки работы сервиса.

Ответ:

- Определение сообщений и сервисов в .proto
- Генерация кода
- Компиляция и запуск приложения
- Тестирование потокового взаимодействия

 Шаг 10:

Миссия 10

Расположите этапы оптимизации производительности gRPC-потокового приложения:

1. Профилирование и сбор метрик
2. Анализ узких мест и проблем
3. Внедрение улучшений и изменений
4. Ретестирование и проверка эффективности

Подсказка: Помните, что оптимизация — это цикл постоянного улучшения.

Ответ:

- Профилирование и сбор метрик
- Анализ узких мест и проблем
- Внедрение улучшений и изменений
- Ретестирование и проверка эффективности

 Урок 7 - Викторина 2

 Шаг 01:

Миссия 1

Сопоставьте типы gRPC стриминга с их описаниями.

Ответ:

- Клиентский стриминг
- Серверный стриминг
- Двухнаправленный стриминг

- Unary вызов

1. Клиент отправляет поток сообщений, сервер отвечает один раз.
2. Сервер отправляет поток сообщений в ответ на один запрос клиента.
3. Обе стороны одновременно обмениваются потоками сообщений.
4. Клиент отправляет один запрос и получает один ответ.

Подсказка: Подумайте, кто инициирует поток и в какую сторону идут сообщения.

- **Клиентский стриминг** — Клиент отправляет поток сообщений, сервер отвечает один раз.
- **Серверный стриминг** — Сервер отправляет поток сообщений в ответ на один запрос клиента.
- **Двунаправленный стриминг** — Обе стороны одновременно обмениваются потоками сообщений.
- **Unary вызов** — Клиент отправляет один запрос и получает один ответ.

Шаг 02:

Миссия 2

Команда обсуждает преимущества стриминга. Сопоставьте термин с его объяснением.

Ответ:

- Flow Control
 - Backpressure
 - Latency
 - Buffering
1. Механизм управления потоком данных для предотвращения перегрузки.
 2. Сигнал от получателя, чтобы отправитель замедлил передачу.
 3. Время задержки между отправкой и получением сообщения.
 4. Временное хранение данных для сглаживания передачи.

Подсказка: Вспомните, как поток данных регулируется для стабильности.

- **Flow Control** — Механизм управления потоком данных для предотвращения перегрузки.
- **Backpressure** — Сигнал от получателя, чтобы отправитель замедлил передачу.
- **Latency** — Время задержки между отправкой и получением сообщения.
- **Buffering** — Временное хранение данных для сглаживания передачи.

Шаг 03:

Миссия 3

При настройке потокового взаимодействия в gRPC, сопоставьте проблемные ситуации с возможными решениями.

Ответ:

- Потеря сообщений из-за высокой нагрузки
 - Задержки при передаче больших объемов данных
 - Блокировка клиента из-за ожидания ответа
 - Неэффективное использование сетевых ресурсов
1. Использование flow control и backpressure
 2. Сегментация данных и оптимизация буферов
 3. Асинхронная обработка сообщений
 4. Компрессия и сжатие потоков

Подсказка: Вспомните, как управлять нагрузкой и оптимизировать обмен данными.

- **Потеря сообщений из-за высокой нагрузки** — Использование flow control и backpressure
- **Задержки при передаче больших объемов данных** — Сегментация данных и оптимизация буферов
- **Блокировка клиента из-за ожидания ответа** — Асинхронная обработка сообщений
- **Неэффективное использование сетевых ресурсов** — Компрессия и сжатие потоков

🔗 Шаг 04:

Миссия 4

Расположите этапы настройки двунаправленного gRPC стриминга в правильном порядке.

1. Определение сервиса и потоковых методов в proto-файле
2. Реализация сервера с поддержкой обработки потоковых сообщений
3. Настройка клиента для отправки и приема сообщений асинхронно
4. Тестирование обмена сообщениями и обработка ошибок

Подсказка: Сначала описываем интерфейс, потом реализуем, подключаем клиента и тестируем.

Ответ:

- Определение сервиса и потоковых методов в proto-файле
- Реализация сервера с поддержкой обработки потоковых сообщений
- Настройка клиента для отправки и приема сообщений асинхронно
- Тестирование обмена сообщениями и обработка ошибок

🔗 Шаг 05:

Миссия 5

Упорядочьте этапы оптимизации производительности стримингового сервиса в gRPC.

1. Сбор метрик и мониторинг задержек
2. Идентификация узких мест и проблем с буферизацией
3. Внедрение flow control и компрессии
4. Проведение нагрузочного тестирования

Подсказка: Для улучшения производительности сначала собираем данные, затем анализируем и исправляем.

Ответ:

- Сбор метрик и мониторинг задержек
- Идентификация узких мест и проблем с буферизацией
- Внедрение flow control и компрессии
- Проведение нагрузочного тестирования

🔗 Шаг 06:

Миссия 6

Упорядочьте шаги обработки ошибок в двунаправленном gRPC стриминге.

1. Обработка ошибок на сервере и генерация статусов
2. Обработка ошибок на клиенте с повторными попытками или отменой
3. Логирование и трассировка ошибок для диагностики
4. Информирование пользователя о статусе операции

Подсказка: Сначала сервер формирует ошибки, клиент их обрабатывает, затем фиксируем и показываем пользователю.

Ответ:

- Обработка ошибок на сервере и генерация статусов
- Обработка ошибок на клиенте с повторными попытками или отменой
- Логирование и трассировка ошибок для диагностики
- Информирование пользователя о статусе операции

Модуль 4 - Middleware и Interceptors

 **Урок 1 - Логирование запросов/ответов**

 **Урок 2 - Метрики и OpenTelemetry**

 **Урок 3 - Ретраи, таймауты, circuit breaker**

 **Урок 4 - Аутентификация (JWT, mTLS)**

 **Урок 5 -  Практика добавить авторизацию и метрики**

 **Урок 6 - Викторина 1**

 **Шаг 01:**

Миссия 1

Вы команда разработчиков gRPC-приложения, и вам нужно добавить логирование всех входящих запросов через middleware. Что лучше всего использовать для этого?

Подсказка: Подумайте, как перехватить вызовы на одном уровне, не меняя логику бизнес-методов.

Ответ:

- **Unary interceptor для логирования входящих запросов**
- Добавить логирование в каждый метод сервиса
- Создать отдельный клиент для логирования

 **Шаг 02:**

Миссия 2

Ваша команда столкнулась с проблемой: некоторые gRPC вызовы занимают слишком много времени, и нужно реализовать таймауты на уровне middleware. Что позволит сделать это наиболее эффективно?

Подсказка: Подумайте, какой компонент может обрабатывать запросы до попадания в бизнес-логику.

Ответ:

- **Серверный interceptor, который контролирует время выполнения запроса**
- Таймаут в каждом методе вручную
- Использовать клиентские таймауты на уровне вызова

 **Шаг 03:**

Миссия 3

Вы хотите добавить аутентификацию пользователей через middleware в gRPC. Как лучше организовать проверку токена в запросах?

Подсказка: Обратите внимание, что аутентификация должна быть прозрачной для основной бизнес-логики.

Ответ:

- **Реализовать interceptor, который проверяет токен и блокирует запрос при ошибке**
- Проверять токен внутри каждого метода сервиса
- Отправлять токен на клиент и ждать подтверждения
- Игнорировать токены и доверять клиенту

Шаг 04:

Миссия 4

Команда обсуждает типы interceptors в gRPC. Сопоставьте тип interceptor с его основным применением:

Ответ:

- Unary interceptor
- Stream interceptor
- Client interceptor
- Server interceptor

Определения:

1. Обрабатывает одиночные запросы и ответы на сервере
2. Обрабатывает потоки данных (streaming) на сервере
3. Перехватывает вызовы на стороне клиента
4. Перехватывает вызовы на стороне сервера

Подсказка: Вспомните разницу между unary и streaming вызовами и где выполняются interceptors.

- **Unary interceptor** — Обрабатывает одиночные запросы и ответы на сервере
- **Stream interceptor** — Обрабатывает потоки данных (streaming) на сервере
- **Client interceptor** — Перехватывает вызовы на стороне клиента
- **Server interceptor** — Перехватывает вызовы на стороне сервера

Шаг 05:

Миссия 5

Сопоставьте middleware-компоненты и их функции в gRPC приложении:

Ответ:

- Логирование
- Аутентификация
- Обработка ошибок
- Таймауты

Функции:

1. Проверка подлинности пользователя или токена
2. Запись информации о вызовах для отладки и аудита
3. Прерывание запросов при превышении времени ожидания
4. Перехват и корректная обработка исключений

Подсказка: Каждый middleware выполняет отдельную задачу, подумайте, что именно нужно для каждой из них.

- **Логирование** — Запись информации о вызовах для отладки и аудита
- **Аутентификация** — Проверка подлинности пользователя или токена
- **Обработка ошибок** — Перехват и корректная обработка исключений
- **Таймауты** — Прерывание запросов при превышении времени ожидания

🔮 Шаг 06:

Миссия 6

Ваша команда разрабатывает цепочку interceptor-ов. Сопоставьте порядок вызова interceptor-ов при одном `map` вызове:

Ответ:

- Первый interceptor
- Второй interceptor
- Метод сервиса
- Ответ возвращается обратно

Пояснения:

1. Interceptor, который первым обрабатывает запрос
2. Interceptor, который следующий в цепочке
3. Основная бизнес-логика метода
4. Ответ проходит через все interceptor-ы в обратном порядке

Подсказка: Подумайте о стеке вызовов и порядке вызова interceptor-ов.

- **Первый interceptor** — Interceptor, который первым обрабатывает запрос
- **Второй interceptor** — Interceptor, который следующий в цепочке
- **Метод сервиса** — Основная бизнес-логика метода
- **Ответ возвращается обратно** — Ответ проходит через все interceptor-ы в обратном порядке

🔮 Шаг 07:

Миссия 7

Расположите этапы создания кастомного interceptor-а в правильном порядке:

1. Реализация функции interceptor-а с нужной логикой
2. Регистрация interceptor-а при инициализации сервера
3. Тестирование interceptor-а на разных сценариях
4. Документирование кода и возможностей interceptor-а

Подсказка: Сначала нужно написать код, затем подключить, проверить и оформить.

Ответ:

- Реализация функции interceptor-а с нужной логикой
- Регистрация interceptor-а при инициализации сервера
- Тестирование interceptor-а на разных сценариях
- Документирование кода и возможностей interceptor-а

🔮 Шаг 08:

Миссия 8

Расставьте этапы обработки gRPC-запроса с middleware в правильном порядке:

1. Запрос поступает на сервер
2. Middleware-interceptor перехватывает запрос
3. Проверяется аутентификация и авторизация
4. Выполняется бизнес-логика метода
5. Ответ проходит через middleware обратно к клиенту
6. Ответ отправляется клиенту

Подсказка: Представьте цепочку обработки запроса на сервере и обратный путь ответа.

Ответ:

- Запрос поступает на сервер
- Middleware-interceptor перехватывает запрос
- Проверяется аутентификация и авторизация
- Выполняется бизнес-логика метода
- Ответ проходит через middleware обратно к клиенту
- Ответ отправляется клиенту

Шаг 09:

Миссия 9

Расположите шаги оптимизации middleware цепочки в правильном порядке:

1. Идентификация узких мест в производительности
2. Профилирование interceptor-ов
3. Оптимизация или удаление тяжелых interceptor-ов
4. Повторное тестирование производительности

Подсказка: Сначала выявляем проблемы, потом профилируем, оптимизируем и проверяем результат.

Ответ:

- Идентификация узких мест в производительности
- Профилирование interceptor-ов
- Оптимизация или удаление тяжелых interceptor-ов
- Повторное тестирование производительности

Шаг 10:

Миссия 10

Расположите порядок действий для интеграции нескольких middleware в gRPC сервер:

1. Создание отдельных interceptor-ов для каждой задачи
2. Объединение interceptor-ов в цепочку
3. Регистрация цепочки interceptor-ов при запуске сервера
4. Проверка совместимости и тестирование всей цепочки

Подсказка: Каждый middleware отвечает за свою задачу, но все должны работать в связке.

Ответ:

- Создание отдельных interceptor-ов для каждой задачи
- Объединение interceptor-ов в цепочку
- Регистрация цепочки interceptor-ов при запуске сервера
- Проверка совместимости и тестирование всей цепочки

Урок 7 - Викторина 2

Шаг 01:

Миссия 1

Наша команда внедряет простой interceptor для логирования каждого запроса в gRPC-сервисе. Что из ниже перечисленного является основной задачей такого interceptor?

Подсказка: Подумайте, зачем нам перехватывать запросы на уровне middleware.

Ответ:

- **Отслеживать и логировать все входящие вызовы сервиса**
- Изменять логику обработки бизнес-запроса
- Хранить состояние пользователя между вызовами
- Отправлять уведомления клиенту напрямую

Шаг 02:

Миссия 2

В нашем проекте middleware используется для аутентификации пользователей. Какой из вариантов лучше всего описывает роль interceptor в этом случае?

Подсказка: Middleware должен проверять права доступа до передачи запроса в бизнес-логику.

Ответ:

- **Проверять токен авторизации и блокировать неавторизованные запросы**
- Обрабатывать ошибки после выполнения запроса
- Выполнять бизнес-логику сервиса
- Передавать запросы без изменений

Шаг 03:

Миссия 3

В команде обсуждают, зачем использовать chaining interceptor — цепочку промежуточных обработчиков. Выберите главное преимущество такого подхода.

Подсказка: Подумайте о структуре кода и расширяемости.

Ответ:

- **Позволяет разделить ответственность и добавить новые функции без изменения существующих**
- Ускоряет выполнение запроса за счет параллельности
- Отключает все проверки безопасности
- Сокращает размер бинарника приложения

Шаг 04:

Миссия 4

Наша команда внедряет middleware для трассировки и мониторинга. Сопоставьте термины и их роли в interceptor:

Ответ:

- Unary interceptor
- Stream interceptor
- Server interceptor

- Client interceptor

1. Обрабатывает одиночные запросы и ответы
2. Работает с потоковыми вызовами
3. Перехватывает запросы на сервере
4. Перехватывает запросы на стороне клиента

Подсказка: Обратите внимание, где именно работает interceptor.

- **Unary interceptor** — Обрабатывает одиночные запросы и ответы
- **Stream interceptor** — Работает с потоковыми вызовами
- **Server interceptor** — Перехватывает запросы на сервере
- **Client interceptor** — Перехватывает запросы на стороне клиента

Шаг 05:

Миссия 5

Команда разбирается с этапами обработки interceptor. Сопоставьте этапы с их описанием:

Ответ:

- Pre-processing
 - Invocation
 - Post-processing
 - Error handling
1. Подготовка и проверка запроса перед выполнением
 2. Выполнение основной логики сервиса
 3. Обработка ответа и логирование
 4. Обработка ошибок и исключений

Подсказка: Проанализируйте последовательность шагов при вызове interceptor.

- **Pre-processing** — Подготовка и проверка запроса перед выполнением
- **Invocation** — Выполнение основной логики сервиса
- **Post-processing** — Обработка ответа и логирование
- **Error handling** — Обработка ошибок и исключений

Шаг 06:

Миссия 6

В процессе разработки middleware для авторизации возникли термины. Сопоставьте их:

Ответ:

- Authentication
 - Authorization
 - Context Propagation
 - Metadata
1. Процесс проверки личности пользователя
 2. Определение прав доступа пользователя
 3. Передача данных в цепочке вызовов
 4. Дополнительная информация в запросе

Подсказка: Подумайте, как информация движется между клиентом и сервером.

- **Authentication** — Процесс проверки личности пользователя

- **Authorization** — Определение прав доступа пользователя
- **Context Propagation** — Передача данных в цепочке вызовов
- **Metadata** — Дополнительная информация в запросе

Шаг 07:

Миссия 7

Расположите в правильном порядке этапы создания кастомного interceptor для логирования в gRPC:

1. Реализация логики логирования
2. Регистрация interceptor в сервере
3. Определение интерфейса interceptor
4. Тестирование interceptor

Подсказка: Подумайте о жизненном цикле разработки компонента.

Ответ:

- Определение интерфейса interceptor
- Реализация логики логирования
- Регистрация interceptor в сервере
- Тестирование interceptor

Модуль 5 - Архитектура и масштабирование (Middle-ready)

Урок 1 - gRPC в микросервисах

Урок 2 - gRPC-gateway для REST

Урок 3 - Балансировка round-robin, Consul, Envoy

Урок 4 - Поднятие сервисов в Kubernetes

Урок 5 - Практика запустить несколько сервисов через Kubernetes + Envoy

Урок 6 - Викторина 1

Шаг 01:

Миссия 1

В проекте используются несколько сервисов с gRPC. Сопоставьте термины с их описаниями.

Термины:

Ответ:

- Proto-файл
- Stub (клиент)
- Сервер gRPC
- Interceptor

Определения:

1. Файл, описывающий структуры данных и методы сервиса в protobuf

2. Промежуточный компонент для обработки запросов/ответов (например, для логирования или аутентификации)
3. Клиентская библиотека для вызова удалённых методов
4. Программа, реализующая серверную часть gRPC-сервиса

Подсказка: Обратите внимание на роль каждого компонента в gRPC-экосистеме.

- **Proto-файл** — Файл, описывающий структуры данных и методы сервиса в protobuf
- **Interceptor** — Промежуточный компонент для обработки запросов/ответов (например, для логирования или аутентификации)
- **Stub (клиент)** — Клиентская библиотека для вызова удалённых методов
- **Сервер gRPC** — Программа, реализующая серверную часть gRPC-сервиса

Шаг 02:

Миссия 2

Команда проектирует масштабируемую систему. Сопоставьте подходы к масштабированию с их описаниями.

Подходы:

Ответ:

- Горизонтальное масштабирование
- Вертикальное масштабирование
- Балансировка нагрузки
- Кэширование

Определения:

1. Добавление большего количества серверов для обработки нагрузки
2. Увеличение ресурсов одного сервера (CPU, RAM)
3. Распределение запросов между несколькими экземплярами сервиса
4. Хранение часто используемых данных для быстрого доступа

Подсказка: Подумайте, как разные способы влияют на производительность и отказоустойчивость.

- **Горизонтальное масштабирование** — Добавление большего количества серверов для обработки нагрузки
- **Вертикальное масштабирование** — Увеличение ресурсов одного сервера (CPU, RAM)
- **Балансировка нагрузки** — Распределение запросов между несколькими экземплярами сервиса
- **Кэширование** — Хранение часто используемых данных для быстрого доступа

Шаг 03:

Миссия 3

Для повышения отказоустойчивости и масштабируемости gRPC-сервиса сопоставьте техники и их цели.

Техники:

Ответ:

- Load Balancer
- Service Discovery
- Health Checking
- Timeouts и Retries

Цели:

1. Автоматический мониторинг доступности сервисов
2. Распределение запросов между доступными сервисами
3. Автоматическое обнаружение новых или удалённых сервисов
4. Предотвращение зависания вызовов и повтор попыток при ошибках

Подсказка: Подумайте, как система должна управлять состоянием и распределять нагрузку.

- **Load Balancer** — Распределение запросов между доступными сервисами
- **Service Discovery** — Автоматическое обнаружение новых или удалённых сервисов
- **Health Checking** — Автоматический мониторинг доступности сервисов
- **Timeouts и Retries** — Предотвращение зависания вызовов и повтор попыток при ошибках

Шаг 04:

Миссия 4

Задача: Расположите этапы оптимизации производительности gRPC-сервиса в правильном порядке:

1. Профилирование и мониторинг узких мест
2. Использование эффективного формата данных (protobuf)
3. Внедрение кэширования часто запрашиваемых данных
4. Настройка таймаутов и ограничение повторных вызовов
5. Оптимизация сетевых настроек и использование HTTP/2

Подсказка:

Оптимизация начинается с анализа проблем, затем идут базовые улучшения и контроль повторов.

Ответ:

- Профилирование и мониторинг узких мест
- Использование эффективного формата данных (protobuf)
- Внедрение кэширования часто запрашиваемых данных
- Настройка таймаутов и ограничение повторных вызовов
- Оптимизация сетевых настроек и использование HTTP/2

Шаг 05:

Миссия 5

Задача: Расставьте шаги масштабирования gRPC-сервиса для поддержки растущей нагрузки:

1. Добавление новых инстансов сервиса
2. Настройка балансировщика нагрузки
3. Мониторинг производительности и логирование
4. Оптимизация кода и запросов
5. Внедрение механизма отказоустойчивости

Подсказка:

Масштабирование — это непрерывный процесс: добавляем ресурсы, распределяем нагрузку и улучшаем качество.

Ответ:

- Добавление новых инстансов сервиса
- Настройка балансировщика нагрузки

- Мониторинг производительности и логирование
- Оптимизация кода и запросов
- Внедрение механизма отказоустойчивости

Шаг 06:

Миссия 6

Задача: Отсортируйте действия при отладке проблемы с задержками в gRPC-сервисе:

1. Сбор метрик и логов
2. Анализ данных и выявление узких мест
3. Тестирование гипотез и исправление ошибок
4. Проверка сетевого соединения и настроек сервера
5. Внедрение улучшений и мониторинг результатов

Подсказка:

Отладка начинается с диагностики, затем анализ, исправления и проверка изменений.

Ответ:

- Сбор метрик и логов
- Анализ данных и выявление узких мест
- Тестирование гипотез и исправление ошибок
- Проверка сетевого соединения и настроек сервера
- Внедрение улучшений и мониторинг результатов

Шаг 07:

Миссия 7

Задача: Расположите этапы внедрения аутентификации и авторизации в gRPC-приложении:

1. Определение требований безопасности
2. Выбор и интеграция механизма аутентификации (например, JWT, mTLS)
3. Реализация interceptors для проверки прав доступа
4. Тестирование безопасности и нагрузочное тестирование
5. Мониторинг безопасности и обновление политик

Подсказка:

Безопасность — это планирование, внедрение, проверка и постоянное улучшение.

Ответ:

- Определение требований безопасности
- Выбор и интеграция механизма аутентификации (например, JWT, mTLS)
- Реализация interceptors для проверки прав доступа
- Тестирование безопасности и нагрузочное тестирование
- Мониторинг безопасности и обновление политик

Урок 7 - Викторина 2

Шаг 01:

Миссия 1

Сопоставьте понятия с их описаниями, связанными с архитектурой gRPC:

Ответ:

- HTTP/2
 - Протокол Buffers
 - Балансировщик нагрузки
 - Версионирование API
1. Высокопроизводительный двоичный формат сериализации сообщений.
 2. Механизм для распределения входящих запросов между серверами.
 3. Протокол транспортного уровня с поддержкой мультиплексирования потоков.
 4. Процесс управления изменениями интерфейса без нарушения клиентов.

Подсказка: Подумайте, зачем нужен каждый элемент для стабильной и масштабируемой системы.

- **HTTP/2** — Протокол транспортного уровня с поддержкой мультиплексирования потоков.
- **Протокол Buffers** — Высокопроизводительный двоичный формат сериализации сообщений.
- **Балансировщик нагрузки** — Механизм для распределения входящих запросов между серверами.
- **Версионирование API** — Процесс управления изменениями интерфейса без нарушения клиентов.

Шаг 02:

Миссия 2

Сопоставьте типы масштабирования с их характеристиками:

Ответ:

- Горизонтальное масштабирование
 - Вертикальное масштабирование
 - Масштабирование с помощью кэширования
 - Масштабирование с помощью шардирования
1. Увеличение ресурсов одного сервера.
 2. Добавление новых серверов в кластер.
 3. Разделение данных на части для параллельной обработки.
 4. Использование памяти для ускорения доступа к данным.

Подсказка: Каждый способ помогает системе работать быстрее, но по-разному.

- **Горизонтальное масштабирование** — Добавление новых серверов в кластер.
- **Вертикальное масштабирование** — Увеличение ресурсов одного сервера.
- **Масштабирование с помощью кэширования** — Использование памяти для ускорения доступа к данным.
- **Масштабирование с помощью шардирования** — Разделение данных на части для параллельной обработки.

Шаг 03:

Миссия 3

Сопоставьте типы потоковой передачи gRPC с их описаниями:

Ответ:

- Unary RPC
- Server streaming RPC

- Client streaming RPC
- Bidirectional streaming RPC

1. Один запрос – один ответ.
2. Клиент отправляет поток сообщений, сервер отвечает одним сообщением.
3. Сервер отправляет поток сообщений в ответ на один запрос клиента.
4. Клиент и сервер обмениваются потоками сообщений одновременно.

Подсказка: Вспомните про типы вызовов в gRPC, они отличаются направлением потоков.

- **Unary RPC** — Один запрос – один ответ.
- **Server streaming RPC** — Сервер отправляет поток сообщений в ответ на один запрос клиента.
- **Client streaming RPC** — Клиент отправляет поток сообщений, сервер отвечает одним сообщением.
- **Bidirectional streaming RPC** — Клиент и сервер обмениваются потоками сообщений одновременно.

Шаг 04:

Миссия 4

Расположите этапы оптимизации gRPC-сервиса для масштабирования:

1. Профилирование и измерение узких мест.
2. Реализация балансировщика нагрузки.
3. Внедрение кэширования ответов.
4. Добавление горизонтального масштабирования.

Подсказка: Сначала нужно понять, где проблема, потом её устранить.

Ответ:

- Профилирование и измерение узких мест.
- Реализация балансировщика нагрузки.
- Внедрение кэширования ответов.
- Добавление горизонтального масштабирования.

Шаг 05:

Миссия 5

Расположите шаги, необходимые для добавления поддержки версионирования в gRPC API:

1. Создание новой версии protobuf-сообщений.
2. Обновление серверной логики для поддержки обеих версий.
3. Информирование клиентов о новой версии API.
4. Плавный переход клиентов на новую версию.

Подсказка: Версионирование должно проходить без сбоев для пользователей.

Ответ:

- Создание новой версии protobuf-сообщений.
- Обновление серверной логики для поддержки обеих версий.
- Информирование клиентов о новой версии API.
- Плавный переход клиентов на новую версию.

Шаг 06:

Миссия 6

Упорядочьте этапы обработки gRPC-запроса на сервере:

1. Получение запроса и десериализация protobuf-сообщения.
2. Выполнение бизнес-логики.
3. Сериализация ответа в protobuf.
4. Отправка ответа клиенту.

Подсказка: Следуйте логике обработки данных с момента получения до отправки.

Ответ:

- Получение запроса и десериализация protobuf-сообщения.
- Выполнение бизнес-логики.
- Сериализация ответа в protobuf.
- Отправка ответа клиенту.

Шаг 07:

Миссия 7

Расположите шаги построения отказоустойчивого gRPC-сервиса:

1. Мониторинг и логирование ошибок.
2. Реализация повторных попыток и таймаутов.
3. Деплой нескольких экземпляров сервиса.
4. Использование балансировщика нагрузки для распределения трафика.

Подсказка: Отказоустойчивость строится поэтапно, начиная с инфраструктуры.

Ответ:

- Деплой нескольких экземпляров сервиса.
- Использование балансировщика нагрузки для распределения трафика.
- Реализация повторных попыток и таймаутов.
- Мониторинг и логирование ошибок.

Модуль 6 - Производительность и оптимизация

Урок 1 - Connection pooling

Урок 2 - Сжатие сообщений (gzip, snappy)

Урок 3 - Минимизация .proto (enum vs string, repeated vs map)

Урок 4 - Benchmarking и профилирование

Урок 5 - Практика замерить скорость сервиса до и после оптимизации

Урок 6 - Викторина 1

Шаг 01:

Миссия 1

Сопоставьте понятия и их роли в производительности gRPC-сервисов:

Ответ:

- HTTP/2
 - Protobuf
 - Балансировщик нагрузки
 - Поточковая передача
1. Двоичный формат сжатия и сериализации сообщений.
 2. Протокол с мультиплексированием и эффективной передачей.
 3. Распределяет запросы между серверами для масштабируемости.
 4. Позволяет передавать данные непрерывным потоком для уменьшения задержек.

Подсказка: Каждый элемент помогает повысить эффективность и скорость.

- **HTTP/2** — Протокол с мультиплексированием и эффективной передачей.
- **Protobuf** — Двоичный формат сжатия и сериализации сообщений.
- **Балансировщик нагрузки** — Распределяет запросы между серверами для масштабируемости.
- **Поточковая передача** — Позволяет передавать данные непрерывным потоком для уменьшения задержек.

 Шаг 02:

Миссия 2

Сопоставьте виды масштабирования с их описаниями:

Ответ:

- Горизонтальное
 - Вертикальное
 - Кэширование
 - Шардирование
1. Добавление серверов в кластер.
 2. Увеличение ресурсов одного сервера.
 3. Хранение часто используемых данных в памяти.
 4. Разделение данных на части для параллельной обработки.

Подсказка: Каждый способ помогает системе работать быстрее, но по-разному.

- **Горизонтальное** — Добавление серверов в кластер.
- **Вертикальное** — Увеличение ресурсов одного сервера.
- **Кэширование** — Хранение часто используемых данных в памяти.
- **Шардирование** — Разделение данных на части для параллельной обработки.

 Шаг 03:

Миссия 3

Сопоставьте типы gRPC-вызовов с их характеристиками:

Ответ:

- Unary RPC
- Server streaming
- Client streaming

- Bidirectional streaming

1. **Один запрос** — один ответ.
2. Сервер отправляет поток сообщений после запроса клиента.
3. Клиент отправляет поток сообщений, сервер отвечает один раз.
4. Клиент и сервер обмениваются потоками сообщений одновременно.

Подсказка: Потоки могут идти в разных направлениях.

- **Unary RPC** — Один запрос — один ответ.
- **Server streaming** — Сервер отправляет поток сообщений после запроса клиента.
- **Client streaming** — Клиент отправляет поток сообщений, сервер отвечает один раз.
- **Bidirectional streaming** — Клиент и сервер обмениваются потоками сообщений одновременно.

Шаг 04:

Миссия 4

Расположите этапы оптимизации gRPC-сервиса для повышения производительности:

1. Анализ узких мест с помощью профилирования.
2. Реализация балансировщика нагрузки.
3. Внедрение кэширования данных.
4. Добавление горизонтального масштабирования.

Подсказка: Начинайте с анализа, потом улучшайте инфраструктуру.

Ответ:

- Анализ узких мест с помощью профилирования.
- Реализация балансировщика нагрузки.
- Внедрение кэширования данных.
- Добавление горизонтального масштабирования.

Шаг 05:

Миссия 5

Расположите шаги внедрения версионирования в gRPC API:

1. Создание новой версии protobuf-сообщений.
2. Обновление серверной логики для поддержки обеих версий.
3. Информирование клиентов о новой версии API.
4. Плавный переход клиентов на новую версию.

Подсказка: Обеспечьте совместимость и постепенный переход.

Ответ:

- Создание новой версии protobuf-сообщений.
- Обновление серверной логики для поддержки обеих версий.
- Информирование клиентов о новой версии API.
- Плавный переход клиентов на новую версию.

Урок 7 - Викторина 2

Шаг 01:

Миссия 1

Ваша команда заметила, что gRPC-сервис отвечает медленно при большом количестве запросов. Что из этого поможет быстро выявить узкое место?

Подсказка: Иногда стоит начать с простого измерения времени.

Ответ:

- **Использовать профайлер для замера времени обработки запросов**
- Добавить больше логов в каждом методе
- Увеличить количество серверов без анализа
- Изменить версию protobuf

Шаг 02:

Миссия 2

Ваш коллега рекомендует включить компрессию для уменьшения объема данных в gRPC. Какие компромиссы стоит учитывать?

Подсказка: Компрессия уменьшает трафик, но может добавить нагрузку.

Ответ:

- **Повышение нагрузки на CPU из-за сжатия и распаковки**
- Значительное увеличение размера данных
- Появление дополнительных ошибок из-за сжатия
- Сокращение времени на обработку запроса

Шаг 03:

Миссия 3

При проектировании потокового gRPC-сервиса, что поможет минимизировать задержки передачи данных?

Подсказка: Помните, что буферизация может влиять на отклик.

Ответ:

- **Использовать меньшие буферы и чаще отправлять данные**
- Отправлять данные большими блоками для уменьшения количества пакетов
- Увеличить таймауты на стороне клиента
- Отключить keep-alive соединения

Шаг 04:

Миссия 4

Сопоставьте типы оптимизаций в gRPC с их описаниями:

Ответ:

- Использование сжатия
- Кэширование ответов
- Параллельные запросы
- Пакетирование сообщений

Описание:

1. Уменьшение количества сетевых вызовов за счёт объединения сообщений
2. Снижение объёма передаваемых данных с помощью алгоритмов сжатия
3. Повторное использование данных без повторных вызовов
4. Выполнение нескольких запросов одновременно для ускорения работы

Подсказка: Внимательно прочитайте, что именно делает каждый метод.

- **Использование сжатия** — Снижение объёма передаваемых данных с помощью алгоритмов сжатия
- **Кэширование ответов** — Повторное использование данных без повторных вызовов
- **Параллельные запросы** — Выполнение нескольких запросов одновременно для ускорения работы
- **Пакетирование сообщений** — Уменьшение количества сетевых вызовов за счёт объединения сообщений

Шаг 05:

Миссия 5

Ваша команда выбирает методы оптимизации для gRPC-сервисов. Сопоставьте проблему и подход к её решению:

Ответ:

- Высокая задержка на сетевом уровне
- Перегрузка CPU на сервере
- Чрезмерное потребление памяти
- Частые блокировки при обработке запросов

Решения:

1. Внедрение сжатия и пакетирования сообщений
2. Оптимизация алгоритмов и профилирование CPU
3. Использование потоков и асинхронности
4. Перераспределение нагрузки и оптимизация кэширования

Подсказка: Сопоставьте тип проблемы с наиболее логичным способом её решения.

- **Высокая задержка на сетевом уровне** — Внедрение сжатия и пакетирования сообщений
- **Перегрузка CPU на сервере** — Оптимизация алгоритмов и профилирование CPU
- **Чрезмерное потребление памяти** — Перераспределение нагрузки и оптимизация кэширования
- **Частые блокировки при обработке запросов** — Использование потоков и асинхронности

Шаг 06:

Миссия 6

Сопоставьте виды метрик производительности gRPC с их значениями:

Ответ:

- Latency
- Throughput
- Error rate
- CPU load

Определения:

1. Количество успешных запросов в единицу времени
2. Время ответа сервиса на запрос
3. Доля неудачных запросов среди всех
4. Процент использования процессорного времени

Подсказка: Внимательно подумайте, какие метрики что измеряют.

- **Latency** — Время ответа сервиса на запрос
- **Throughput** — Количество успешных запросов в единицу времени
- **Error rate** — Доля неудачных запросов среди всех
- **CPU load** — Процент использования процессорного времени

Шаг 07:

Миссия 7

Расположите этапы профилирования и оптимизации gRPC-сервиса в правильном порядке:

1. Анализ собранных метрик и выявление узких мест
2. Измерение производительности до изменений
3. Внесение изменений и оптимизаций в код
4. Повторное измерение производительности и сравнение результатов

Подсказка: Чтобы улучшить, сначала надо измерить, потом менять, и снова измерять.

Ответ:

- Измерение производительности до изменений
- Анализ собранных метрик и выявление узких мест
- Внесение изменений и оптимизаций в код
- Повторное измерение производительности и сравнение результатов

Шаг 08:

Миссия 8

Упорядочьте шаги внедрения кэширования в gRPC-сервисе:

1. Выбор подходящего места для кэширования (клиент, сервер, промежуточный слой)
2. Настройка политики обновления кэша и TTL
3. Реализация и тестирование кэширования
4. Мониторинг эффективности кэша и корректировка параметров

Подсказка: Важно сначала понять, где кэш нужен, а потом уже настраивать его поведение.

Ответ:

- Выбор подходящего места для кэширования (клиент, сервер, промежуточный слой)
- Настройка политики обновления кэша и TTL
- Реализация и тестирование кэширования
- Мониторинг эффективности кэша и корректировка параметров

Шаг 09:

Миссия 9

Отсортируйте рекомендации по оптимизации потоковой передачи в gRPC:

1. Минимизируйте размер сообщений
2. Используйте асинхронную обработку на клиенте и сервере
3. Используйте компрессию при необходимости
4. Избегайте избыточных вызовов и повторной передачи данных

Подсказка: Подумайте, что стоит оптимизировать сначала для скорости и нагрузки, а что потом.

Ответ:

- Минимизируйте размер сообщений
- Используйте асинхронную обработку на клиенте и сервере
- Используйте компрессию при необходимости
- Избегайте избыточных вызовов и повторной передачи данных

 Шаг 10:

Миссия 10

Расположите шаги оптимизации нагрузочного тестирования gRPC-сервиса:

1. Определение ключевых сценариев нагрузки и целевых метрик
2. Настройка среды тестирования и генерация трафика
3. Анализ результатов тестов и выявление узких мест
4. Внедрение оптимизаций и повторное тестирование

Подсказка: Эффективное тестирование — это циклический процесс измерения, анализа и улучшения.

Ответ:

- Определение ключевых сценариев нагрузки и целевых метрик
- Настройка среды тестирования и генерация трафика
- Анализ результатов тестов и выявление узких мест
- Внедрение оптимизаций и повторное тестирование

 Модуль 7 - CI/CD и тестирование

 Урок 1 - Генерация кода на этапе CI

 Урок 2 - Contract tests для gRPC

 Урок 3 - Интеграционные тесты

 Урок 4 - Mock-сервисы

 Урок 5 - Практика GitHub Actions pipeline для gRPC

 Урок 6 - Викторина 1

 Шаг 01:

Миссия 1

В нашей команде решили внедрить CI/CD для gRPC-приложений. Что является ключевым преимуществом использования CI/CD в разработке?

Подсказка: Подумайте, как CI/CD помогает сократить время между изменениями кода и их доставкой.

Ответ:

- **Автоматизация тестирования и быстрая доставка изменений в продакшн**
- Увеличение времени на ручную проверку кода
- Отсрочка релизов на неопределенный срок
- Удаление необходимости в тестах

🔮 Шаг 02:

Миссия 2

Разработчики столкнулись с проблемой: при изменениях в protobuf-файлах CI/CD пайплайн не запускается автоматически. Что могло быть причиной?

Подсказка: Подумайте, какие файлы должны быть триггерами для запуска пайплайна.

Ответ:

- **Настройка триггеров в CI/CD не учитывает изменения в protobuf-файлах**
- CI/CD не поддерживает gRPC-приложения
- Protobuf-файлы не влияют на сборку и тесты
- CI/CD требует ручного запуска для protobuf

🔮 Шаг 03:

Миссия 3

Какая из стратегий тестирования лучше всего подходит для проверки новых версий gRPC-сервисов в CI/CD?

Подсказка: Подумайте, что важно проверить, чтобы избежать регрессий и сохранить совместимость.

Ответ:

- **Юнит-тесты для логики сервиса**
- **Интеграционные тесты с реальными gRPC вызовами**
- Тесты интерфейса пользователя
- Ручное тестирование без автоматизации

🔮 Шаг 04:

Миссия 4

Сопоставьте этапы CI/CD с их описаниями в контексте разработки gRPC-приложений.

Этапы:

Ответ:

- Сборка
- Тестирование
- Деплой
- Мониторинг

Описание:

1. Автоматическая отправка приложения на сервер или кластер
2. Проверка кода на ошибки и корректность работы сервисов
3. Отслеживание состояния приложения после выпуска
4. Компиляция и генерация кода из protobuf

Подсказка: Каждый этап имеет четкую роль в цепочке доставки.

- **Сборка** — Компиляция и генерация кода из protobuf
- **Тестирование** — Проверка кода на ошибки и корректность работы сервисов
- **Деплой** — Автоматическая отправка приложения на сервер или кластер
- **Мониторинг** — Отслеживание состояния приложения после выпуска

🔮 Шаг 05:

Миссия 5

Сопоставьте инструменты с их назначением в gRPC CI/CD процессах.

Инструменты:

Ответ:

- Protobuf Compiler (protoc)
- Docker
- Jenkins
- Prometheus

Назначения:

1. Автоматизация сборки и развертывания
2. Мониторинг и сбор метрик
3. Контейнеризация приложений
4. Генерация исходного кода из protobuf

Подсказка: Подумайте, что именно делает каждый инструмент и какую задачу он решает.

- **Protobuf Compiler (protoc)** — Генерация исходного кода из protobuf
- **Docker** — Контейнеризация приложений
- **Jenkins** — Автоматизация сборки и развертывания
- **Prometheus** — Мониторинг и сбор метрик

🔮 Шаг 06:

Миссия 6

Сопоставьте термины с их описаниями в контексте тестирования gRPC приложений.

Термины:

Ответ:

- Mock
- Stub
- End-to-End тестирование
- Load тестирование

Определения:

1. Тестирование производительности под высокой нагрузкой
2. Тестирование полной цепочки взаимодействия сервисов
3. Имитатор поведения сервиса для изоляции тестов
4. Заглушка для возврата предсказуемых данных

Подсказка: Разберитесь в разнице между изоляцией и интеграцией.

- **Mock** — Имитатор поведения сервиса для изоляции тестов
- **Stub** — Заглушка для возврата предсказуемых данных
- **End-to-End тестирование** — Тестирование полной цепочки взаимодействия сервисов
- **Load тестирование** — Тестирование производительности под высокой нагрузкой

🔮 Шаг 07:

Миссия 7

Вопрос: Упорядочьте этапы создания и деплоя gRPC-сервиса в CI/CD с учётом правильной последовательности.

1. Написание тестов и проверка локально
2. Автоматическая сборка и генерация кода из protobuf
3. Деплой на тестовый сервер
4. Мониторинг состояния сервиса после релиза
5. Выпуск релиза в продакшн

Подсказка: От разработки к тестированию, затем развертыванию и контролю.

Ответ:

- Написание тестов и проверка локально
- Автоматическая сборка и генерация кода из protobuf
- Деплой на тестовый сервер
- Мониторинг состояния сервиса после релиза
- Выпуск релиза в продакшн

Шаг 08:

Миссия 8

Вопрос: Расставьте в правильном порядке действия для отладки CI/CD пайплайна gRPC-приложения после неудачного билда.

1. Анализ логов сборки
2. Изменение кода или конфигурации пайплайна
3. Запуск сборки заново
4. Проверка корректности тестов и их покрытия

Подсказка: Начинаем с поиска причины ошибки, потом исправляем и проверяем изменения.

Ответ:

- Анализ логов сборки
- Изменение кода или конфигурации пайплайна
- Запуск сборки заново
- Проверка корректности тестов и их покрытия

Шаг 09:

Миссия 9

Вопрос: Расположите шаги автоматизации тестирования gRPC-сервисов по правильному порядку.

1. Настройка тестовой среды с сервисами-заглушками
2. Написание тестов для gRPC методов
3. Запуск тестов в CI/CD пайплайне
4. Анализ результатов и исправление ошибок

Подсказка: Автоматизация начинается с подготовки окружения и заканчивается разбором результатов.

Ответ:

- Настройка тестовой среды с сервисами-заглушками
- Написание тестов для gRPC методов
- Запуск тестов в CI/CD пайплайне

- Анализ результатов и исправление ошибок

Шаг 10:

Миссия 10

Вопрос: Упорядочьте этапы настройки мониторинга и алертинга для gRPC-приложения в продакшне.

1. Выбор метрик для мониторинга (ошибки, задержки, нагрузки)
2. Настройка сбора метрик с помощью Prometheus или аналогов
3. Определение порогов для алертов
4. Конфигурация системы оповещений (email, Slack, SMS)

Подсказка: Мониторинг начинается с выбора данных, заканчивается уведомлением команды.

Ответ:

- Выбор метрик для мониторинга (ошибки, задержки, нагрузки)
- Настройка сбора метрик с помощью Prometheus или аналогов
- Определение порогов для алертов
- Конфигурация системы оповещений (email, Slack, SMS)

Урок 7 - Викторина 2

Шаг 01:

Миссия 1

Сопоставьте типы тестов с их основными задачами в gRPC-приложениях.

Типы тестов:

Ответ:

- Unit тесты
- Интеграционные тесты
- End-to-end тесты
- Нагрузочные тесты

Задачи:

1. Проверка отдельных функций и методов в изоляции
2. Проверка взаимодействия между сервисами через gRPC
3. Проверка всей системы от начала до конца в реальных сценариях
4. Оценка производительности и устойчивости под нагрузкой

Подсказка: Подумайте, на каком уровне тесты проверяют разные аспекты приложения.

- **Unit тесты** — Проверка отдельных функций и методов в изоляции
- **Интеграционные тесты** — Проверка взаимодействия между сервисами через gRPC
- **End-to-end тесты** — Проверка всей системы от начала до конца в реальных сценариях
- **Нагрузочные тесты** — Оценка производительности и устойчивости под нагрузкой

Шаг 02:

Миссия 2

Команда обсуждает ошибки, найденные в CI/CD. Сопоставьте тип ошибки с возможной причиной.

Типы ошибок:

Ответ:

- Сбой сборки
- Провал теста
- Проблемы с сетью
- Несоответствие версий protobuf

Возможные причины:

1. Ошибки компиляции или зависимости
2. Неправильная логика или баг в коде
3. Проблемы связи между сервисами в CI
4. Изменения схемы без синхронизации между сервисами

Подсказка: Подумайте, какие ошибки влияют на сборку, а какие — на взаимодействие сервисов.

- **Сбой сборки** — Ошибки компиляции или зависимости
- **Провал теста** — Неправильная логика или баг в коде
- **Проблемы с сетью** — Проблемы связи между сервисами в CI
- **Несоответствие версий protobuf** — Изменения схемы без синхронизации между сервисами

 Шаг 03:

Миссия 3

Команда анализирует этапы CI/CD для gRPC-сервисов. Сопоставьте этап с его назначением.

Этапы:

Ответ:

- Сборка
- Тестирование
- Деплоймент
- Мониторинг

Назначения:

1. Компиляция protobuf и сборка бинарников
2. Проверка корректности работы кода
3. Развёртывание сервиса в инфраструктуру
4. Отслеживание состояния и производительности сервисов

Подсказка: Вспомните жизненный цикл прохождения кода от написания до эксплуатации.

- **Сборка** — Компиляция protobuf и сборка бинарников
- **Тестирование** — Проверка корректности работы кода
- **Деплоймент** — Развёртывание сервиса в инфраструктуру
- **Мониторинг** — Отслеживание состояния и производительности сервисов

 Шаг 04:

Миссия 4

Расположите шаги процесса CI/CD для gRPC-приложения в правильном порядке:

1. Запуск автоматических тестов
2. Публикация собранных артефактов
3. Сборка и компиляция protobuf файлов

4. Деплой сервисов на тестовое окружение
5. Мониторинг и логирование после деплоя

Подсказка: Обратите внимание, что тесты запускаются после сборки, а мониторинг — после деплоя.

Ответ:

- Сборка и компиляция protobuf файлов
- Запуск автоматических тестов
- Публикация собранных артефактов
- Деплой сервисов на тестовое окружение
- Мониторинг и логирование после деплоя

Шаг 05:

Миссия 5

Расположите шаги отладки ошибок в CI/CD пайплайне gRPC-сервиса:

1. Анализ логов сборки и тестов
2. Проверка конфигураций CI/CD
3. Воспроизведение ошибки локально
4. Исправление кода или конфигураций
5. Запуск повторной сборки и тестов

Подсказка: Начинайте с поиска причины ошибки, затем исправляйте и проверяйте.

Ответ:

- Анализ логов сборки и тестов
- Проверка конфигураций CI/CD
- Воспроизведение ошибки локально
- Исправление кода или конфигураций
- Запуск повторной сборки и тестов

Шаг 06:

Миссия 6

Расположите этапы написания и интеграции новых тестов для gRPC-сервиса:

1. Определение тест-кейсов по спецификации API
2. Написание тестов для каждого кейса
3. Запуск тестов локально и в CI
4. Анализ результатов и исправление багов
5. Документирование тестов и покрытия

Подсказка: Сначала нужно понять, что тестировать, потом проверить, что тесты проходят.

Ответ:

- Определение тест-кейсов по спецификации API
- Написание тестов для каждого кейса
- Запуск тестов локально и в CI
- Анализ результатов и исправление багов
- Документирование тестов и покрытия

Шаг 07:

Миссия 7

Команда строит оптимальный pipeline CI/CD для gRPC-сервисов. Расположите этапы по приоритету выполнения:

1. Проверка синтаксиса и форматирования protobuf
2. Сборка и компиляция кода и protobuf
3. Запуск unit и интеграционных тестов
4. Деплой на staging окружение
5. Автоматический мониторинг и оповещения

Подсказка: Помните, что сначала код должен быть корректным и протестированным, прежде чем деплоить.

Ответ:

- Проверка синтаксиса и форматирования protobuf
- Сборка и компиляция кода и protobuf
- Запуск unit и интеграционных тестов
- Деплой на staging окружение
- Автоматический мониторинг и оповещения

Модуль 8 - Senior-практики отказоустойчивость

Урок 1 - HA и fault tolerance

Урок 2 - Circuit breaker и rate limiting

Урок 3 - gRPC + KafkaRabbitMQ (event-driven)

Урок 4 - Обработка сетевых ошибок и деградация сервиса

Урок 5 - Практика сервис с graceful degradation

Урок 6 - Викторина 1

Шаг 01:

Миссия 1

Ваша команда только что внедрила gRPC-сервис. Какой первый шаг для повышения отказоустойчивости стоит сделать?

Подсказка: Подумайте, как минимизировать простои при сбоях.

Ответ:

- **Настроить автоматический повтор запросов (retry) с экспоненциальной задержкой**
- Игнорировать ошибки и логировать только предупреждения
- Отключить все таймауты для вызовов
- Выключить балансировщик нагрузки, чтобы упростить трассировку

Шаг 02:

Миссия 2

Команда обсуждает, как минимизировать время восстановления при сбое gRPC-сервиса. Что поможет в этом?

Подсказка: Думайте про автоматизацию и мониторинг.

Ответ:

- **Настроить health check и автоматический перезапуск сервисов через оркестратор**
- Реализовать только ручное вмешательство при сбоях
- Отказаться от логирования ошибок для ускорения работы
- Убрать кэширование, чтобы не усложнять систему

Шаг 03:

Миссия 3

В вашем gRPC-приложении резко возросло количество ошибок таймаута. Что первоочередно проверить?

Подсказка: Ошибки таймаута часто связаны с сетевыми задержками или нагрузкой.

Ответ:

- **Проверить настройки таймаутов и сетевую задержку между сервисами**
- Увеличить логирование до максимума, не меняя ничего
- Переключиться на синхронные вызовы без таймаутов
- Отказаться от балансировки нагрузки

Шаг 04:

Миссия 4

Сопоставьте основные практики повышения отказоустойчивости с их описаниями:

Ответ:

- Репликация сервисов
- Автоматический retry
- Circuit Breaker
- Health Checks

Определения:

1. Механизм для прекращения вызовов при частых ошибках, чтобы избежать перегрузки
2. Проверка состояния сервиса для быстрого обнаружения неполадок
3. Копирование данных или сервисов для обеспечения резервирования
4. Повтор запроса в случае неудачи с задержками между попытками

Подсказка: Каждая практика решает конкретную проблему с надежностью.

- **Репликация сервисов** — Копирование данных или сервисов для обеспечения резервирования
- **Автоматический retry** — Повтор запроса в случае неудачи с задержками между попытками
- **Circuit Breaker** — Механизм для прекращения вызовов при частых ошибках, чтобы избежать перегрузки
- **Health Checks** — Проверка состояния сервиса для быстрого обнаружения неполадок

Шаг 05:

Миссия 5

Сопоставьте причины сбоев в gRPC-приложениях с оптимальными решениями:

Ответ:

- Перегрузка сервиса
- Падение одного из микросервисов
- Проблемы с сетью
- Ошибки валидации данных

Решения:

1. Использование балансировщика нагрузки и очередей
2. Реализация retries с backoff и circuit breaker
3. Настройка таймаутов и fallback механизма
4. Добавление валидации и контроль качества данных на уровне API

Подсказка: Подумайте, как каждой проблеме можно противостоять с помощью архитектуры и кода.

- **Перегрузка сервиса** — Использование балансировщика нагрузки и очередей
- **Падение одного из микросервисов** — Реализация retries с backoff и circuit breaker
- **Проблемы с сетью** — Настройка таймаутов и fallback механизма
- **Ошибки валидации данных** — Добавление валидации и контроль качества данных на уровне API

 Шаг 06:

Миссия 6

Сопоставьте типы тестирования с их назначением в контексте отказоустойчивости gRPC-сервисов:

Ответ:

- Load Testing
- Chaos Testing
- Integration Testing
- Unit Testing

Назначения:

1. Проверка работы компонентов в изоляции
2. Проверка реакции системы на сбои и непредвиденные ситуации
3. Проверка взаимодействия компонентов и их отказоустойчивости вместе
4. Измерение производительности под нагрузкой

Подсказка: Каждый вид теста отвечает за отдельный аспект надежности.

- **Load Testing** — Измерение производительности под нагрузкой
- **Chaos Testing** — Проверка реакции системы на сбои и непредвиденные ситуации
- **Integration Testing** — Проверка взаимодействия компонентов и их отказоустойчивости вместе
- **Unit Testing** — Проверка работы компонентов в изоляции

 Шаг 07:

Миссия 7

Расположите этапы построения отказоустойчивого gRPC-сервиса в правильном порядке:

1. Мониторинг и alerting для быстрого обнаружения проблем

2. Реализация retry и circuit breaker в клиенте
3. Деплой нескольких реплик сервиса с балансировкой нагрузки
4. Настройка health check и автоматического восстановления сервисов

Подсказка: Откажитесь от хаотичных действий — стройте систему шаг за шагом.

Ответ:

- Деплой нескольких реплик сервиса с балансировкой нагрузки
- Настройка health check и автоматического восстановления сервисов
- Реализация retry и circuit breaker в клиенте
- Мониторинг и alerting для быстрого обнаружения проблем

Шаг 08:

Миссия 8

Расположите шаги при обработке ошибки таймаута в gRPC-клиенте:

1. Логирование ошибки с контекстом
2. Повтор запроса с увеличением задержки (backoff)
3. Проверка статуса сервиса через health check
4. Возврат fallback-ответа, если retries исчерпаны

Подсказка: Рассмотрите стратегию устойчивости к временным сбоям.

Ответ:

- Логирование ошибки с контекстом
- Проверка статуса сервиса через health check
- Повтор запроса с увеличением задержки (backoff)
- Возврат fallback-ответа, если retries исчерпаны

Шаг 09:

Миссия 9

Расположите этапы внедрения circuit breaker в gRPC-сервис:

1. Определение порогов срабатывания (количество ошибок, таймауты)
2. Переключение в состояние "открыто", блокирующее вызовы
3. Проверка восстановления через ограниченное количество тестовых вызовов
4. Возврат в состояние "закрыто" при успешных вызовах

Подсказка: Circuit breaker защищает систему от лавины ошибок.

Ответ:

- Определение порогов срабатывания (количество ошибок, таймауты)
- Переключение в состояние "открыто", блокирующее вызовы
- Проверка восстановления через ограниченное количество тестовых вызовов
- Возврат в состояние "закрыто" при успешных вызовах

Шаг 10:

Миссия 10

Расположите шаги при организации отказоустойчивого деплоя gRPC-приложения:

1. Развертывание новой версии на ограниченном числе реплик (canary)
2. Мониторинг метрик и логов для обнаружения проблем

3. Плавное переключение всего трафика на новую версию
4. Откат на предыдущую версию при критических ошибках

Подсказка: Безопасность релизов — залог отказоустойчивости.

Ответ:

- Развертывание новой версии на ограниченном числе реплик (canary)
- Мониторинг метрик и логов для обнаружения проблем
- Плавное переключение всего трафика на новую версию
- Откат на предыдущую версию при критических ошибках

Урок 7 - Викторина 2

Шаг 01:

Миссия 1

Команда gRPC-разработчиков заметила, что при нагрузке их сервис часто выдаёт ошибки из-за перегрузки. Что из ниже перечисленного поможет улучшить отказоустойчивость сервиса в первую очередь?

Подсказка: Подумайте о базовых способах защиты от перегрузки.

Ответ:

- **Внедрить механизм ограничения количества одновременных запросов (rate limiting)**
- Увеличить таймауты вызовов на стороне клиента
- Использовать статическую маршрутизацию запросов
- Отключить повторные вызовы (retries)

Шаг 02:

Миссия 2

При внедрении повторных попыток (retries) для gRPC вызовов, что необходимо обязательно учитывать?

Подсказка: Подумайте, что может ухудшить ситуацию при неправильной настройке retries.

Ответ:

- **Использовать экспоненциальный backoff, чтобы избежать лавинообразной нагрузки**
- Устанавливать очень короткие интервалы между попытками
- Не ограничивать количество попыток
- Игнорировать ошибки таймаута и сразу повторять

Шаг 03:

Миссия 3

Что из перечисленного является примером "Circuit Breaker" паттерна в gRPC сервисах?

Подсказка: Это механизм для предотвращения избыточных запросов к упавшему сервису.

Ответ:

- **Прекращение отправки запросов на сервис при превышении количества ошибок за короткий промежуток времени**
- Повторная попытка запроса без ограничений
- Увеличение таймаута для всех запросов
- Отправка всех запросов через балансировщик без учета состояния сервисов

Шаг 04:

Миссия 4

Команда внедряет таймауты в gRPC вызовы. Какой подход обеспечит оптимальную отказоустойчивость?

Подсказка: Слишком большие таймауты могут замедлить реакцию системы, слишком маленькие — вызвать ложные срабатывания.

Ответ:

- **Установить таймаут, немного превышающий среднее время обработки запроса с запасом на пиковые нагрузки**
- Установить максимально короткий таймаут для мгновенного прерывания
- Убрать таймауты и полагаться на балансировщик нагрузки
- Устанавливать таймаут в зависимости от количества запросов за секунду

Шаг 05:

Миссия 5

Сопоставьте паттерны отказоустойчивости с их описанием в контексте gRPC приложений:

Паттерны:

Ответ:

- Retry
- Circuit Breaker
- Timeout
- Load Balancing

Описание:

1. Ограничение времени ожидания ответа сервера, чтобы не блокировать клиента.
2. Автоматический повтор вызова при временных ошибках.
3. Переключение на другой сервер или инстанс для равномерного распределения нагрузки.
4. Отключение вызовов к сервису при превышении ошибок, чтобы дать ему восстановиться.

Подсказка: Каждый паттерн решает конкретную задачу, связанную с надежностью и производительностью.

- **Retry** — Автоматический повтор вызова при временных ошибках.
- **Circuit Breaker** — Отключение вызовов к сервису при превышении ошибок, чтобы дать ему восстановиться.
- **Timeout** — Ограничение времени ожидания ответа сервера, чтобы не блокировать клиента.
- **Load Balancing** — Переключение на другой сервер или инстанс для равномерного распределения нагрузки.

Шаг 06:

Миссия 6

Сопоставьте виды ошибок в gRPC с подходящими стратегиями обработки для отказоустойчивости:

Ошибки:

Ответ:

- UNAVAILABLE
- DEADLINE_EXCEEDED

- INTERNAL
- RESOURCE_EXHAUSTED

Стратегии:

1. Использовать экспоненциальные повторы с ограничением по времени.
2. Скорректировать таймауты и отменить запрос, если превышен лимит времени.
3. Переключиться на другой инстанс или сервис, если возможно.
4. Реализовать контроль нагрузки и rate limiting.

Подсказка: Ошибки связаны с разными проблемами — это определяет подход к их обработке.

- **UNAVAILABLE** — Переключиться на другой инстанс или сервис, если возможно.
- **DEADLINE_EXCEEDED** — Скорректировать таймауты и отменить запрос, если превышен лимит времени.
- **INTERNAL** — Использовать экспоненциальные повторы с ограничением по времени.
- **RESOURCE_EXHAUSTED** — Реализовать контроль нагрузки и rate limiting.

Шаг 07:

Миссия 7

Сопоставьте типы балансировки нагрузки с их преимуществами в gRPC-системах:

Типы балансировки:

Ответ:

- Round Robin
- Least Connections
- Random
- Weighted

Преимущества:

1. Равномерное распределение запросов по серверам независимо от состояния.
2. Переключение на сервер с наименьшей нагрузкой для оптимальной производительности.
3. Используется для простого случайного распределения, уменьшает вероятность перегрузки одного сервера.
4. Позволяет направлять больше трафика на более мощные серверы.

Подсказка: Балансировка помогает распределять нагрузку, учитывая разное состояние и мощность серверов.

- **Round Robin** — Равномерное распределение запросов по серверам независимо от состояния.
- **Least Connections** — Переключение на сервер с наименьшей нагрузкой для оптимальной производительности.
- **Random** — Используется для простого случайного распределения, уменьшает вероятность перегрузки одного сервера.
- **Weighted** — Позволяет направлять больше трафика на более мощные серверы.

Шаг 08:

Миссия 8

Расположите шаги внедрения Circuit Breaker в gRPC клиенте в правильном порядке:

1. Отслеживание ошибок и состояния соединения
2. Открытие цепи (прекращение вызовов) при превышении порога ошибок

3. Переход в состояние "полуоткрытого" для тестирования сервиса
4. Закрытие цепи и возврат к нормальной работе при восстановлении

Подсказка: Circuit Breaker сначала мониторит состояние, затем временно отключает вызовы, проверяет сервис и восстанавливает работу.

Ответ:

- Отслеживание ошибок и состояния соединения
- Открытие цепи (прекращение вызовов) при превышении порога ошибок
- Переход в состояние "полуоткрытого" для тестирования сервиса
- Закрытие цепи и возврат к нормальной работе при восстановлении

 Шаг 09:

Миссия 9

Расположите действия команды для отладки и устранения проблем с отказоустойчивостью gRPC сервиса в правильном порядке:

1. Анализ логов и метрик на предмет повторяющихся ошибок
2. Внедрение или корректировка таймаутов и retries
3. Настройка Circuit Breaker для снижения нагрузки на упавшие сервисы
4. Тестирование и мониторинг изменений в продакшене

Подсказка: Сначала выявляем проблему, затем решаем её пошагово, после чего проверяем результат.

Ответ:

- Анализ логов и метрик на предмет повторяющихся ошибок
- Внедрение или корректировка таймаутов и retries
- Настройка Circuit Breaker для снижения нагрузки на упавшие сервисы
- Тестирование и мониторинг изменений в продакшене

 Шаг 10:

Миссия 10

Расположите в правильном порядке этапы внедрения комплексной отказоустойчивости в gRPC микросервисе:

1. Определение SLA и критичных точек отказа
2. Проектирование и реализация retry и backoff стратегий
3. Добавление Circuit Breaker и мониторинга состояния сервисов
4. Регулярное тестирование на отказоустойчивость и нагрузочное тестирование
5. Внедрение логирования и метрик для оперативного реагирования

Подсказка: Планируйте, реализуйте, отслеживайте и тестируйте для гарантированной надежности.

Ответ:

- Определение SLA и критичных точек отказа
- Проектирование и реализация retry и backoff стратегий
- Добавление Circuit Breaker и мониторинга состояния сервисов
- Регулярное тестирование на отказоустойчивость и нагрузочное тестирование
- Внедрение логирования и метрик для оперативного реагирования

Модуль 9 - Advanced Security

 Урок 1 - mTLS шифрование и валидация клиентов

 Урок 2 - Ротация ключей и сертификатов

 Урок 3 - RBAC и ABAC поверх gRPC

 Урок 4 - Интеграция с OAuth2

 Урок 5 -  Практика настроить mTLS и выдать сертификаты

 Урок 6 - Викторина 1

Шаг 01:

Миссия 1

Сопоставьте методы безопасности с их назначением в gRPC:

Ответ:

- Аутентификация
- Авторизация
- Шифрование
- Аудит

Подсказка: Подумайте, какая задача отвечает за проверку личности, а какая — за права доступа.

- **Аутентификация** — Подтверждение личности пользователя или сервиса
- **Авторизация** — Определение прав доступа пользователя к ресурсам
- **Шифрование** — Защита данных от перехвата и чтения посторонними
- **Аудит** — Отслеживание и запись действий пользователей и сервисов

Шаг 02:

Миссия 2

Сопоставьте уровни доступа с возможностями пользователя в gRPC-сервисе:

Ответ:

- Read-only
- Read-write
- Admin
- Guest

Подсказка: У разных ролей разные возможности — подумайте, кто что может делать.

- **Read-only** — Только просмотр данных без изменений
- **Read-write** — Просмотр и изменение данных
- **Admin** — Полный доступ, включая управление пользователями
- **Guest** — Ограниченный доступ, минимальные права

Шаг 03:

Миссия 3

Сопоставьте угрозы безопасности с методами их предотвращения в gRPC:

Ответ:

- Man-in-the-Middle
- Replay Attack
- DoS-атака
- SQL Injection

Подсказка: Каждая угроза требует своего способа защиты.

- **Man-in-the-Middle** — Использование TLS для шифрования и аутентификации сервера
- **Replay Attack** — Внедрение nonce и проверки времени запроса
- **DoS-атака** — Ограничение числа запросов и защита от перегрузок
- **SQL Injection** — Использование параметризованных запросов и валидация входных данных

 Шаг 04:

Миссия 4

Расположите шаги настройки безопасного gRPC-сервиса с использованием TLS в правильном порядке:

1. Настройка серверного сертификата
2. Создание ключей и сертификатов клиента (опционально)
3. Включение шифрования TLS на сервере
4. Настройка проверки сертификатов на клиенте

Подсказка: Правильный порядок важен для успешного подключения и защиты.

Ответ:

- Настройка серверного сертификата
- Создание ключей и сертификатов клиента (опционально)
- Включение шифрования TLS на сервере
- Настройка проверки сертификатов на клиенте

 Шаг 05:

Миссия 5

Расположите этапы внедрения аутентификации JWT в gRPC-сервисе в правильном порядке:

1. Генерация JWT токена при логине пользователя
2. Передача токена в метаданных каждого gRPC-запроса
3. Проверка и валидация токена на сервере
4. Обработка результата проверки и предоставление доступа

Подсказка: Каждый этап важен для надежной работы аутентификации.

Ответ:

- Генерация JWT токена при логине пользователя
- Передача токена в метаданных каждого gRPC-запроса
- Проверка и валидация токена на сервере
- Обработка результата проверки и предоставление доступа

 Шаг 06:

Миссия 6

Расположите шаги реакции на подозрительный инцидент безопасности в gRPC-сервисе:

1. Выявление подозрительной активности
2. Изоляция и ограничение доступа подозрительного клиента
3. Анализ логов и возможных уязвимостей
4. Принятие мер по устранению угрозы и уведомление команды

Подсказка: От последовательности действий зависит скорость реагирования.

Ответ:

- Выявление подозрительной активности
- Изоляция и ограничение доступа подозрительного клиента
- Анализ логов и возможных уязвимостей
- Принятие мер по устранению угрозы и уведомление команды

Шаг 07:

Миссия 7

Расположите этапы внедрения RBAC (Role-Based Access Control) в gRPC-приложении:

1. Определение ролей и их прав
2. Привязка пользователей к ролям
3. Реализация проверки прав на сервере
4. Тестирование и мониторинг доступа

Подсказка: Контроль доступа строится по четкой модели ролей.

Ответ:

- Определение ролей и их прав
- Привязка пользователей к ролям
- Реализация проверки прав на сервере
- Тестирование и мониторинг доступа

Урок 7 - Викторина 2

Шаг 01:

Миссия 1

Ваша команда внедряет базовую аутентификацию в gRPC сервис. Какой протокол лучше всего использовать для безопасной передачи учетных данных?

Подсказка: Безопасность передачи — ключевой момент при работе с пользователями.

Ответ:

- **TLS (Transport Layer Security)**
- HTTP без шифрования
- FTP

Шаг 02:

Миссия 2

В вашем проекте gRPC-сервис обрабатывает запросы из разных клиентов. Какой метод аутентификации будет более масштабируемым и безопасным?

Подсказка: Думайте о централизованном контроле доступа и токенах.

Ответ:

- **JWT (JSON Web Token)**
- Передача паролей в каждом запросе
- IP-белый список без аутентификации
- Basic Auth без HTTPS

🔮 **Шаг 03:**

Миссия 3

Вы хотите ограничить доступ к gRPC-сервису по ролям. Что лучше использовать для реализации контроля доступа?

Подсказка: Используйте систему с разграничением прав на уровне токенов.

Ответ:

- **RBAC (Role-Based Access Control)**
- Открытый доступ без контроля
- Использование IP-адреса клиента
- Аутентификация по MAC-адресу

🔮 **Шаг 04:**

Миссия 4

Сопоставьте типы атак с их описанием, которые могут угрожать gRPC-сервисам:

Ответ:

- Man-in-the-Middle (MitM)
- Replay Attack
- Denial of Service (DoS)
- SQL Injection

Описание:

1. Повторное использование перехваченных сообщений для обмана сервиса.
2. Внедрение вредоносного кода в запросы для получения доступа к базе.
3. Перехват и изменение данных во время передачи между клиентом и сервером.
4. Перегрузка сервиса чрезмерным количеством запросов, вызывающая отказ в обслуживании.

Подсказка: Обратите внимание на то, что каждая атака использует уникальный метод для достижения своей цели.

- **Man-in-the-Middle (MitM)** — Перехват и изменение данных во время передачи между клиентом и сервером.
- **Replay Attack** — Повторное использование перехваченных сообщений для обмана сервиса.
- **Denial of Service (DoS)** — Перегрузка сервиса чрезмерным количеством запросов, вызывающая отказ в обслуживании.
- **SQL Injection** — Внедрение вредоносного кода в запросы для получения доступа к базе.

🔮 **Шаг 05:**

Миссия 5

Сопоставьте способы защиты gRPC-сервисов с их описанием:

Ответ:

- TLS
- OAuth 2.0
- Rate Limiting
- Input Validation

Описание:

1. Шифрование и проверка целостности данных между клиентом и сервером.
2. Ограничение количества запросов от клиента для предотвращения перегрузок.
3. Проверка и фильтрация данных, полученных от клиента, для предотвращения атак.
4. Механизм авторизации через сторонний сервис с выдачей токенов.

Подсказка: Подумайте, какую задачу решает каждый из методов.

- **TLS** — Шифрование и проверка целостности данных между клиентом и сервером.
- **OAuth 2.0** — Механизм авторизации через сторонний сервис с выдачей токенов.
- **Rate Limiting** — Ограничение количества запросов от клиента для предотвращения перегрузок.
- **Input Validation** — Проверка и фильтрация данных, полученных от клиента, для предотвращения атак.

 Шаг 06:

Миссия 6

Сопоставьте роли в команде gRPC-разработчиков с их основными задачами по безопасности:

Ответ:

- DevOps-инженер
- Backend-разработчик
- QA-инженер
- Security-аналитик

Задачи:

1. Настройка и поддержка безопасной инфраструктуры и CI/CD.
2. Реализация безопасного кода и протоколов аутентификации.
3. Тестирование на уязвимости и проверка безопасности сервисов.
4. Анализ угроз и разработка рекомендаций по защите.

Подсказка: Помните, что безопасность — это командная игра с четким распределением ролей.

- **DevOps-инженер** — Настройка и поддержка безопасной инфраструктуры и CI/CD.
- **Backend-разработчик** — Реализация безопасного кода и протоколов аутентификации.
- **QA-инженер** — Тестирование на уязвимости и проверка безопасности сервисов.
- **Security-аналитик** — Анализ угроз и разработка рекомендаций по защите.

 Шаг 07:

Миссия 7

Расположите шаги настройки безопасного gRPC-сервиса в правильном порядке:

1. Настройка TLS для защиты канала связи
2. Внедрение механизма аутентификации клиентов
3. Определение ролей и прав доступа

4. Тестирование безопасности и проверка уязвимостей
5. Мониторинг и логирование безопасности

Подсказка: Подумайте, что сначала нужно защитить, а что проверять в конце.

Ответ:

- Настройка TLS для защиты канала связи
- Внедрение механизма аутентификации клиентов
- Определение ролей и прав доступа
- Тестирование безопасности и проверка уязвимостей
- Мониторинг и логирование безопасности

Шаг 08:

Миссия 8

Расположите этапы внедрения OAuth 2.0 в gRPC-приложение:

1. Получение и хранение токена доступа
2. Регистрация клиента в авторизационном сервере
3. Передача токена вместе с запросом
4. Проверка токена сервером gRPC
5. Обновление токена по необходимости

Подсказка: Подумайте о жизненном цикле токена и как он применяется при запросах.

Ответ:

- Регистрация клиента в авторизационном сервере
- Получение и хранение токена доступа
- Передача токена вместе с запросом
- Проверка токена сервером gRPC
- Обновление токена по необходимости

Шаг 09:

Миссия 9

Расположите последовательность действий при обнаружении уязвимости в gRPC-сервисе:

1. Оповещение команды и фиксирование проблемы
2. Оценка риска и приоритетности устранения
3. Разработка и тестирование исправления
4. Развертывание обновления в продакшен
5. Мониторинг для предотвращения повторения

Подсказка: Думайте как в реальных условиях — важно быстро и последовательно реагировать.

Ответ:

- Оповещение команды и фиксирование проблемы
- Оценка риска и приоритетности устранения
- Разработка и тестирование исправления
- Развертывание обновления в продакшен
- Мониторинг для предотвращения повторения

Шаг 10:

Миссия 10

Расположите этапы реализации защиты от DoS-атак в gRPC-сервисе:

1. Анализ поведения трафика и выявление аномалий
2. Настройка ограничений по количеству запросов (rate limiting)
3. Блокировка подозрительных IP или клиентов
4. Мониторинг и логирование событий безопасности
5. Проведение регулярного аудита и обновление политик

Подсказка: Помните, что защита — это постоянный процесс, а не одноразовое действие.

Ответ:

- Анализ поведения трафика и выявление аномалий
- Настройка ограничений по количеству запросов (rate limiting)
- Блокировка подозрительных IP или клиентов
- Мониторинг и логирование событий безопасности
- Проведение регулярного аудита и обновление политик

Модуль 10 - Продвинутые паттерны интеграции

Урок 1 - gRPC + GraphQL (federation)

Урок 2 - gRPC-web (браузерные клиенты)

Урок 3 - gRPC + REST гибрид

Урок 4 - API-шлюз (Kong, Ambassador, Istio)

Урок 5 - Практика REST+gRPC единый API для фронтенда

Урок 6 - Викторина 1

Шаг 01:

Миссия 1

Сопоставьте паттерны интеграции с их основными характеристиками:

Ответ:

- Паттерны:
 1. Request-Reply
 2. Publish-Subscribe
 3. Stream Processing
 4. Circuit Breaker
- 1. Обработка постоянного потока данных с минимальной задержкой.
- 2. Отправка запроса и ожидание ответа от сервиса.
- 3. Механизм подписки на события и получение уведомлений.
- 4. Защита от каскадных сбоев путем временного отключения вызовов.

Подсказка: Обратите внимание на то, как каждый паттерн управляет взаимодействием между сервисами.

- **Request-Reply** — Отправка запроса и ожидание ответа от сервиса.
- **Publish-Subscribe** — Механизм подписки на события и получение уведомлений.
- **Stream Processing** — Обработка постоянного потока данных с минимальной задержкой.
- **Circuit Breaker** — Защита от каскадных сбоев путем временного отключения вызовов.

🔮 Шаг 02:

Миссия 2

Сопоставьте виды потоковой передачи данных по gRPC с их особенностями:

Ответ:

- Виды:
 - 1. Unary RPC
 - 2. Server Streaming RPC
 - 3. Client Streaming RPC
 - 4. Bidirectional Streaming RPC
1. Один запрос от клиента и один ответ от сервера.
 2. Клиент отправляет поток данных, сервер отвечает один раз.
 3. Сервер отправляет поток данных в ответ на один запрос клиента.
 4. Двусторонний обмен потоками данных между клиентом и сервером.

Подсказка: Вспомните, кто инициирует и принимает потоки в каждом типе RPC.

- **Unary RPC** — Один запрос от клиента и один ответ от сервера.
- **Server Streaming RPC** — Сервер отправляет поток данных в ответ на один запрос клиента.
- **Client Streaming RPC** — Клиент отправляет поток данных, сервер отвечает один раз.
- **Bidirectional Streaming RPC** — Двусторонний обмен потоками данных между клиентом и сервером.

🔮 Шаг 03:

Миссия 3

Сопоставьте задачи, которые решают паттерны интеграции, с их кратким описанием:

Ответ:

- Задачи:
 - 1. Повышение отказоустойчивости
 - 2. Обеспечение масштабируемости
 - 3. Уменьшение задержек
 - 4. Гарантированная доставка сообщений
1. Использование повторных попыток и Circuit Breaker.
 2. Распределение нагрузки и асинхронная обработка.
 3. Параллельная обработка и потоковые вызовы.
 4. Использование очередей сообщений и подтверждений.

Подсказка: Подумайте, какие механизмы лучше всего подходят для каждой задачи.

- **Повышение отказоустойчивости** — Использование повторных попыток и Circuit Breaker.
- **Обеспечение масштабируемости** — Распределение нагрузки и асинхронная обработка.
- **Уменьшение задержек** — Параллельная обработка и потоковые вызовы.
- **Гарантированная доставка сообщений** — Использование очередей сообщений и подтверждений.

🔮 Шаг 04:

Миссия 4

Расположите шаги реализации паттерна Circuit Breaker в gRPC-сервисе:

1. Обнаружение частых ошибок в ответах сервиса.
2. Временное блокирование запросов к проблемному сервису.
3. Проверка восстановленного состояния сервиса.
4. Переход в нормальный режим после успешных проверок.

Подсказка: Circuit Breaker помогает защитить систему от каскадных отказов.

Ответ:

- Обнаружение частых ошибок в ответах сервиса.
- Временное блокирование запросов к проблемному сервису.
- Проверка восстановленного состояния сервиса.
- Переход в нормальный режим после успешных проверок.

Шаг 05:

Миссия 5

Расположите этапы интеграции потоковой передачи данных между микросервисами с помощью gRPC:

1. Инициация потока клиентом или сервером.
2. Передача серии сообщений по установленному каналу.
3. Обработка и подтверждение полученных сообщений.
4. Закрытие потока после завершения передачи.

Подсказка: Потоковая передача — это последовательный обмен данными без блокировок.

Ответ:

- Инициация потока клиентом или сервером.
- Передача серии сообщений по установленному каналу.
- Обработка и подтверждение полученных сообщений.
- Закрытие потока после завершения передачи.

Шаг 06:

Миссия 6

Расположите этапы интеграции с использованием паттерна Publish-Subscribe в gRPC-архитектуре:

1. Публикация события отправителем.
2. Получение события подписчиком.
3. Обработка события подписчиком.
4. Регистрация подписчиков на определённые темы.

Подсказка: Этот паттерн позволяет разделить отправителей сообщений от получателей.

Ответ:

- Регистрация подписчиков на определённые темы.
- Публикация события отправителем.
- Получение события подписчиком.
- Обработка события подписчиком.

Шаг 07:

Миссия 7

Расположите шаги оптимизации взаимодействия микросервисов через gRPC с использованием асинхронных вызовов:

1. Инициировать асинхронный вызов клиента.
2. Обработать запрос в сервисе без блокировки клиента.
3. Получить результат вызова через callback или Future.
4. Продолжить выполнение клиентского кода без ожидания ответа.

Подсказка: Асинхронность позволяет эффективнее использовать ресурсы и улучшает отзывчивость системы.

Ответ:

- Инициировать асинхронный вызов клиента.
- Обработать запрос в сервисе без блокировки клиента.
- Продолжить выполнение клиентского кода без ожидания ответа.
- Получить результат вызова через callback или Future.

Урок 7 - Викторина 2

Шаг 01:

Миссия 1

Команда обсуждает, какие паттерны интеграции подходят для разных сценариев в gRPC. Сопоставьте тип интеграции с его описанием.

Типы интеграции:

Ответ:

- Unary
- Client Streaming
- Server Streaming
- Bidirectional Streaming

Описание:

1. Клиент отправляет один запрос и получает один ответ.
2. Клиент отправляет поток сообщений, сервер отвечает одним сообщением.
3. Клиент отправляет один запрос, сервер отправляет поток сообщений.
4. Клиент и сервер обмениваются потоками сообщений одновременно.

Подсказка: Подумайте, как в каждом типе меняется направление и количество сообщений.

- **Unary** — Клиент отправляет один запрос и получает один ответ.
- **Client Streaming** — Клиент отправляет поток сообщений, сервер отвечает одним сообщением.
- **Server Streaming** — Клиент отправляет один запрос, сервер отправляет поток сообщений.
- **Bidirectional Streaming** — Клиент и сервер обмениваются потоками сообщений одновременно.

Шаг 02:

Миссия 2

При внедрении gRPC в команду возникли вопросы по обработке ошибок. Сопоставьте код ошибки с его описанием.

Коды ошибок:

Ответ:

- **DEADLINE_EXCEEDED**
- **UNAVAILABLE**
- **INVALID_ARGUMENT**
- **NOT_FOUND**

Описание:

1. Время ожидания ответа сервера истекло.
2. Сервис временно недоступен.
3. Переданы некорректные параметры запроса.
4. Запрашиваемый ресурс не найден.

Подсказка: Подумайте, какой код ошибки соответствует времени ожидания, а какой — проблемам с параметрами.

- **DEADLINE_EXCEEDED** — Время ожидания ответа сервера истекло.
- **UNAVAILABLE** — Сервис временно недоступен.
- **INVALID_ARGUMENT** — Переданы некорректные параметры запроса.
- **NOT_FOUND** — Запрашиваемый ресурс не найден.

Шаг 03:

Миссия 4

В команде идет обсуждение этапов настройки gRPC сервера. Расположите шаги в правильном порядке:

1. Определение сервиса и сообщений в proto-файле
2. Генерация серверного и клиентского кода
3. Настройка параметров сервера (порты, безопасность)
4. Запуск сервера и ожидание входящих запросов

Подсказка: Подумайте, как начинается любой gRPC проект — с описания интерфейса, потом генерация кода и настройка сервера.

Ответ:

- Определение сервиса и сообщений в proto-файле
- Генерация серверного и клиентского кода
- Настройка параметров сервера (порты, безопасность)
- Запуск сервера и ожидание входящих запросов

Шаг 04:

Миссия 5

Разработчики обсуждают этапы отладки и тестирования gRPC сервиса. Расположите шаги в правильном порядке:

1. Написание unit-тестов для gRPC методов
2. Запуск сервиса локально для интеграционных тестов
3. Мониторинг и анализ логов сервера
4. Исправление ошибок и повторный запуск тестов

Подсказка: Подумайте, что сначала создаются тесты, потом запускается сервис, после исправления ошибок важно проверить результаты.

Ответ:

- Написание unit-тестов для gRPC методов
- Запуск сервиса локально для интеграционных тестов
- Мониторинг и анализ логов сервера
- Исправление ошибок и повторный запуск тестов

Шаг 05:

Миссия 6

Команда внедряет паттерн Circuit Breaker для повышения устойчивости gRPC приложения. Расположите действия, которые выполняет Circuit Breaker, в правильном порядке:

1. Отслеживание количества ошибок и времени отклика
2. Открытие цепи и отказ в обслуживании запросов при превышении порогов
3. Периодическое пробное восстановление связи
4. Закрытие цепи при восстановлении нормальной работы

Подсказка: Подумайте, как Circuit Breaker помогает предотвратить каскадные сбои и восстанавливается после ошибок.

Ответ:

- Отслеживание количества ошибок и времени отклика
- Открытие цепи и отказ в обслуживании запросов при превышении порогов
- Периодическое пробное восстановление связи
- Закрытие цепи при восстановлении нормальной работы

Шаг 06:

Миссия 3

Команда решает, какие методы аутентификации лучше использовать в gRPC. Сопоставьте метод аутентификации с его описанием.

Методы:

Ответ:

- SSL/TLS
- Token-based Authentication
- OAuth2
- API Key

Описание:

1. Шифрование канала для защиты данных.
2. Использование временных маркеров для доступа к API.
3. Авторизация через сторонний сервис с использованием токенов доступа.
4. Уникальный ключ для идентификации клиента.

Подсказка: Обратите внимание, что некоторые методы обеспечивают безопасность канала, а другие — контроль доступа.

- **SSL/TLS** — Шифрование канала для защиты данных.
- **Token-based Authentication** — Использование временных маркеров для доступа к API.
- **OAuth2** — Авторизация через сторонний сервис с использованием токенов доступа.

- **API Key** — Уникальный ключ для идентификации клиента.

Шаг 07:

Миссия 7

Команда организует CI/CD для gRPC проекта. Расположите этапы деплоя в правильном порядке:

1. Сборка и тестирование проекта
2. Публикация артефактов и образов
3. Деплой на тестовый сервер для проверки
4. Деплой на продакшн после успешного тестирования

Подсказка: Подумайте о том, как проходят стандартные циклы выпуска ПО — от сборки до развертывания на продакшн.

Ответ:

- Сборка и тестирование проекта
- Публикация артефактов и образов
- Деплой на тестовый сервер для проверки
- Деплой на продакшн после успешного тестирования

Модуль 11 - Observability & Monitoring (Senior-ready)

Урок 1 - Tracing (Jaeger, Zipkin, OpenTelemetry)

Урок 2 - Centralized logging (ELKGraylog)

Урок 3 - Прометей-метрики для gRPC

Урок 4 - Алерты и SLOSLA

Урок 5 - Практика собрать дашборд метрик в Grafana

Урок 6 - Викторина 1

Шаг 01:

Миссия 1

Сопоставьте типы метрик gRPC с их назначением.

Типы метрик:

Ответ:

- Счетчик (Counter)
- Гистограмма (Histogram)
- Gauge
- Summary

Назначения:

1. Измеряет текущее значение (например, использование памяти)
2. Считает количество событий (например, количество запросов)
3. Измеряет распределение значений (например, время ответа)
4. Подсчитывает статистику по выборке, включая квантиль

Подсказка: Обратите внимание, что счетчик только увеличивается, а Gauge может как увеличиваться, так и уменьшаться.

- **Счетчик (Counter)** — Считает количество событий (например, количество запросов)
- **Гистограмма (Histogram)** — Измеряет распределение значений (например, время ответа)
- **Gauge** — Измеряет текущее значение (например, использование памяти)
- **Summary** — Подсчитывает статистику по выборке, включая квантиль

🔮 Шаг 02:

Миссия 2

Сопоставьте типы логов gRPC с их примерами.

Типы логов:

Ответ:

- Информационные (Info)
- Отладочные (Debug)
- Предупреждения (Warning)
- Ошибки (Error)

Примеры:

1. Ошибки при соединении с базой данных
2. Старт сервиса или успешное выполнение задачи
3. Подробные данные о шагах выполнения для разработки
4. Потенциальные проблемы, которые не останавливают работу

Подсказка: Ошибки сигнализируют о сбоях, а предупреждения — о возможных проблемах.

- **Информационные (Info)** — Старт сервиса или успешное выполнение задачи
- **Отладочные (Debug)** — Подробные данные о шагах выполнения для разработки
- **Предупреждения (Warning)** — Потенциальные проблемы, которые не останавливают работу
- **Ошибки (Error)** — Ошибки при соединении с базой данных

🔮 Шаг 03:

Миссия 3

Сопоставьте компоненты системы мониторинга gRPC с их функциями.

Компоненты:

Ответ:

- Prometheus
- Grafana
- Jaeger
- OpenTelemetry

Функции:

1. Визуализация метрик
2. Сбор и хранение метрик
3. Трассировка запросов
4. Инструмент для сбора данных о мониторинге и трассировке

Подсказка: Разные инструменты выполняют разные задачи, от сбора до визуализации.

- **Prometheus** — Сбор и хранение метрик
- **Grafana** — Визуализация метрик
- **Jaeger** — Трассировка запросов
- **OpenTelemetry** — Инструмент для сбора данных о мониторинге и трассировке

Шаг 04:

Миссия 4

Расположите этапы настройки мониторинга gRPC-сервиса в правильном порядке:

1. Добавление экспортера метрик в сервис
2. Настройка сбора и хранения метрик (например, Prometheus)
3. Создание дашборда для визуализации (например, Grafana)
4. Анализ и реагирование на полученные данные

Подсказка: Мониторинг начинается с данных, заканчивается их анализом.

Ответ:

- Добавление экспортера метрик в сервис
- Настройка сбора и хранения метрик (например, Prometheus)
- Создание дашборда для визуализации (например, Grafana)
- Анализ и реагирование на полученные данные

Шаг 05:

Миссия 5

Расположите шаги диагностики медленных gRPC-вызовов:

1. Сбор метрик задержек
2. Определение аномалий по времени ответа
3. Изучение трассировок запросов с высоким временем выполнения
4. Оптимизация выявленных узких мест

Подсказка: Сначала соберите данные, потом анализируйте, и только потом исправляйте.

Ответ:

- Сбор метрик задержек
- Определение аномалий по времени ответа
- Изучение трассировок запросов с высоким временем выполнения
- Оптимизация выявленных узких мест

Шаг 06:

Миссия 6

Расположите шаги интеграции OpenTelemetry в gRPC-приложение:

1. Установка SDK OpenTelemetry
2. Инструментирование кода gRPC для сбора метрик и трассировок
3. Конфигурация экспорта данных в backend (Jaeger, Prometheus и др.)
4. Проверка корректности данных и мониторинг

Подсказка: Сначала подготовьте инструменты, затем добавьте сбор данных, после настройте экспорт и проверьте.

Ответ:

- Установка SDK OpenTelemetry
- Инструментирование кода gRPC для сбора метрик и трассировок
- Конфигурация экспорта данных в backend (Jaeger, Prometheus и др.)
- Проверка корректности данных и мониторинг

🔗 Шаг 07:

Вопрос 7

Что является принципом "чистого кода" при рефакторинге?

Подсказка: Уменьшение дублирования — ключ к чистоте кода.

Ответ:

- **Вынос повторяющегося кода в отдельные функции**
- Добавление длинных комментариев
- Смешивание логики и интерфейса
- Использование непонятных сокращений

📖 Урок 7 - Викторина 2

🔗 Шаг 01:

Миссия 1

Ваша команда изучает типы метрик gRPC. Сопоставьте типы с их назначением:

Типы метрик:

Ответ:

- Counter
- Gauge
- Histogram
- Summary

Назначения:

1. Текущее значение (например, количество активных соединений)
2. Счётчик событий (например, количество запросов)
3. Распределение значений (например, время ответа)
4. Статистика с квантилями (например, задержки)

Подсказка: Счётчик только растёт, Gauge может увеличиваться и уменьшаться.

- **Counter** — Счётчик событий (например, количество запросов)
- **Gauge** — Текущее значение (например, количество активных соединений)
- **Histogram** — Распределение значений (например, время ответа)
- **Summary** — Статистика с квантилями (например, задержки)

🔗 Шаг 02:

Миссия 2

Команда анализирует логи gRPC. Сопоставьте типы логов с их описаниями:

Типы логов:

Ответ:

- Info
- Debug
- Warning
- Error

Описание:

1. Ошибки соединения с базой данных
2. Подробные данные для отладки
3. Потенциальные проблемы без остановки сервиса
4. Запуск сервиса или успешное завершение задачи

Подсказка: Ошибки — сбои, предупреждения — подозрительные ситуации.

- **Info** — Запуск сервиса или успешное завершение задачи
- **Debug** — Подробные данные для отладки
- **Warning** — Потенциальные проблемы без остановки сервиса
- **Error** — Ошибки соединения с базой данных

 Шаг 03:

Миссия 3

Ваша команда внедряет инструменты мониторинга. Сопоставьте компоненты с их функциями:

Компоненты:

Ответ:

- Prometheus
- Grafana
- Jaeger
- OpenTelemetry

Функции:

1. Сбор и хранение метрик
2. Визуализация метрик
3. Трассировка запросов
4. Сбор данных мониторинга и трассировки

Подсказка: Каждый инструмент решает свою задачу: от сбора до визуализации.

- **Prometheus** — Сбор и хранение метрик
- **Grafana** — Визуализация метрик
- **Jaeger** — Трассировка запросов
- **OpenTelemetry** — Сбор данных мониторинга и трассировки

 Шаг 04:

Миссия 4

Расположите этапы настройки мониторинга gRPC-сервиса в правильном порядке:

1. Добавление экспортера метрик в сервис
2. Настройка сбора и хранения метрик (Prometheus)
3. Создание дашборда для визуализации (Grafana)
4. Анализ и реагирование на данные мониторинга

Подсказка: Мониторинг начинается с получения данных и заканчивается их анализом.

Ответ:

- Добавление экспортера метрик в сервис
- Настройка сбора и хранения метрик (Prometheus)
- Создание дашборда для визуализации (Grafana)
- Анализ и реагирование на данные мониторинга

 **Шаг 05:**

Миссия 5

Расположите шаги диагностики медленных gRPC-вызовов:

1. Сбор метрик задержек
2. Обнаружение аномалий по времени ответа
3. Изучение трассировок долгих запросов
4. Оптимизация выявленных узких мест

Подсказка: Сначала соберите данные, потом проанализируйте, потом исправьте.

Ответ:

- Сбор метрик задержек
- Обнаружение аномалий по времени ответа
- Изучение трассировок долгих запросов
- Оптимизация выявленных узких мест

Модуль 12 - Архитектура уровня Senior

 **Урок 1 - Monorepo vs Polyrepo для gRPC-сервисов**

 **Урок 2 - Domain-driven design + gRPC**

 **Урок 3 - Версионирование API**

 **Урок 4 - Backwardforward compatibility**

 **Урок 5 -  Практика спроектировать многомодульное приложение**

 **Урок 6 - Викторина 1**

 **Шаг 01:**

Миссия 1

Сопоставьте термин с правильным описанием из мира gRPC:

Подсказка: Вспомните, что означают разные элементы в gRPC архитектуре.

Ответ:

- Protocol Buffers
 - Unary RPC
 - Streaming RPC
 - Deadline
1. Формат сериализации и описание структуры сообщений.
 2. Вызов сервиса с одним запросом и одним ответом.

3. Вызов, где сообщения передаются потоками между клиентом и сервером.

4. Временное ограничение на выполнение вызова.

- **Protocol Buffers** — Формат сериализации и описание структуры сообщений.
- **Unary RPC** — Вызов сервиса с одним запросом и одним ответом.
- **Streaming RPC** — Вызов, где сообщения передаются потоками между клиентом и сервером.
- **Deadline** — Временное ограничение на выполнение вызова.

Шаг 02:

Миссия 2

Сопоставьте типы gRPC-статусов с их назначением:

Подсказка: Ошибки могут быть связаны с клиентом, сервером или сетью.

Ответ:

- UNAVAILABLE
- INVALID_ARGUMENT
- DEADLINE_EXCEEDED
- OK

1. Сервер недоступен или сеть временно не работает.
2. Запрос содержит неверные данные.
3. Истекло время ожидания ответа.
4. Запрос выполнен успешно.

- **UNAVAILABLE** — Сервер недоступен или сеть временно не работает.
- **INVALID_ARGUMENT** — Запрос содержит неверные данные.
- **DEADLINE_EXCEEDED** — Истекло время ожидания ответа.
- **OK** — Запрос выполнен успешно.

Шаг 03:

Миссия 3

Команда хочет улучшить производительность gRPC-сервиса. Сопоставьте методы оптимизации с их описаниями:

Подсказка: Некоторые оптимизации касаются кэширования, другие — управления соединениями.

Ответ:

- Keepalive
- Load balancing
- Caching
- Compression

1. Поддержание активного соединения для предотвращения таймаутов.
2. Распределение запросов между несколькими серверами.
3. Хранение данных для повторного использования без повторных запросов.
4. Сжатие сообщений для уменьшения объема передаваемых данных.

- **Keepalive** — Поддержание активного соединения для предотвращения таймаутов.
- **Load balancing** — Распределение запросов между несколькими серверами.
- **Caching** — Хранение данных для повторного использования без повторных запросов.
- **Compression** — Сжатие сообщений для уменьшения объема передаваемых данных.

Шаг 04:

Миссия 4

Расположите этапы разработки gRPC-сервиса в правильном порядке:

Подсказка: Сначала описываем API, потом генерируем код, реализуем логику и тестируем.

1. Реализация серверной логики на основе сгенерированных интерфейсов.
2. Описание API и сообщений в protobuf-файлах.
3. Генерация кода клиентских и серверных заглушек.
4. Написание и запуск интеграционных тестов.

Ответ:

- Описание API и сообщений в protobuf-файлах.
- Генерация кода клиентских и серверных заглушек.
- Реализация серверной логики на основе сгенерированных интерфейсов.
- Написание и запуск интеграционных тестов.

Шаг 05:

Миссия 5

У команды возникли проблемы с деградацией сервиса под нагрузкой. Расположите правильные шаги для устранения проблемы:

Подсказка: Начните с мониторинга, затем анализируйте, оптимизируйте и проверяйте изменения.

1. Оптимизация тяжелых запросов и лимитов ресурсов.
2. Мониторинг производительности и сбор метрик.
3. Анализ узких мест на основе собранных данных.
4. Повторное тестирование после внесенных изменений.

Ответ:

- Мониторинг производительности и сбор метрик.
- Анализ узких мест на основе собранных данных.
- Оптимизация тяжелых запросов и лимитов ресурсов.
- Повторное тестирование после внесенных изменений.

Шаг 06:

Миссия 6

Команда планирует внедрить потоковую передачу (streaming) в gRPC. Расположите этапы её реализации:

Подсказка: Планируйте API, обновите protobuf, реализуйте логику и протестируйте.

1. Добавление стриминговых методов в protobuf-файлы.
2. Разработка серверной и клиентской логики для работы с потоками.
3. Написание тестов, проверяющих корректность потоковой передачи.
4. Обсуждение и дизайн API с командой.

Ответ:

- Обсуждение и дизайн API с командой.
- Добавление стриминговых методов в protobuf-файлы.
- Разработка серверной и клиентской логики для работы с потоками.

- Написание тестов, проверяющих корректность потоковой передачи.

Шаг 07:

Миссия 7

Команда строит систему с несколькими gRPC-сервисами. Расположите действия по интеграции и обеспечению отказоустойчивости:

Подсказка: Сначала проектируем взаимодействие, потом реализуем, мониторим и настраиваем резервные сценарии.

1. Проектирование контрактов взаимодействия между сервисами.
2. Реализация вызовов с учётом таймаутов и повторов.
3. Настройка мониторинга и логирования для сервисов.
4. Внедрение fallback-стратегий и circuit breaker.

Ответ:

- Проектирование контрактов взаимодействия между сервисами.
- Реализация вызовов с учётом таймаутов и повторов.
- Настройка мониторинга и логирования для сервисов.
- Внедрение fallback-стратегий и circuit breaker.

Урок 7 - Викторина 2

Шаг 01:

Миссия 1

Команда начала проектировать gRPC-сервис. Какой формат сообщений обеспечивает gRPC по умолчанию?

Подсказка: Это бинарный формат, обеспечивающий компактность и скорость.

Ответ:

- **Protocol Buffers**
- JSON
- XML
- YAML

Шаг 02:

Миссия 2

Разработчики обсуждают виды взаимодействия в gRPC. Какой тип вызова позволяет клиенту и серверу обмениваться сообщениями одновременно?

Подсказка: Подумайте о диалоге, где обе стороны говорят одновременно.

Ответ:

- **Двунаправленный стриминг (Bidirectional Streaming)**
- Однонаправленный стриминг (Client Streaming)
- Одноразовый вызов (Unary)
- Серверный стриминг (Server Streaming)

Шаг 03:

Миссия 3

Команда обсуждает, как обрабатывать ошибки в gRPC. Какой код статуса указывает на отсутствие прав у клиента?

Подсказка: Этот код соответствует классическому HTTP 403.

Ответ:

- **PERMISSION_DENIED**
- UNAUTHENTICATED
- NOT_FOUND
- FAILED_PRECONDITION

Шаг 04:

Миссия 4

В команде появилась задача разделить большой protobuf-файл на несколько. Как правильно организовать импорты в .proto файлах?

Подсказка: Импорт в protobuf похож на импорт в любом другом языке программирования.

Ответ:

- **Использовать директиву import с относительным путем к нужному файлу**
- Копировать весь код из одного файла в другой вручную
- Указывать путь к скомпилированному файлу .pb.go или .pb.cc

Шаг 05:

Миссия 5

Сопоставьте виды gRPC вызовов с их описаниями.

Подсказка: Внимательно рассмотрите, кто и как отправляет сообщения в каждом типе.

Ответ:

- Unary
 - Server Streaming
 - Client Streaming
 - Bidirectional Streaming
1. Клиент отправляет одно сообщение, сервер отвечает одним.
 2. Клиент отправляет одно сообщение, сервер отправляет поток сообщений.
 3. Клиент отправляет поток сообщений, сервер отвечает одним сообщением.
 4. Клиент и сервер обмениваются потоками сообщений одновременно.
- **Unary** — Клиент отправляет одно сообщение, сервер отвечает одним.
 - **Server Streaming** — Клиент отправляет одно сообщение, сервер отправляет поток сообщений.
 - **Client Streaming** — Клиент отправляет поток сообщений, сервер отвечает одним сообщением.
 - **Bidirectional Streaming** — Клиент и сервер обмениваются потоками сообщений одновременно.

Шаг 06:

Миссия 6

Сопоставьте термины gRPC с их значениями.

Подсказка: Обратите внимание, что некоторые термины связаны с производительностью, а другие — с архитектурой.

Ответ:

- Channel
- Stub
- Interceptor
- Deadline

1. Объект для управления соединением с сервером.
2. Автоматически сгенерированный клиент для вызова методов сервиса.
3. Механизм перехвата и обработки вызовов (например, для логирования или аутентификации).
4. Временное ограничение на выполнение RPC.

- **Channel** — Объект для управления соединением с сервером.
- **Stub** — Автоматически сгенерированный клиент для вызова методов сервиса.
- **Interceptor** — Механизм перехвата и обработки вызовов (например, для логирования или аутентификации).
- **Deadline** — Временное ограничение на выполнение RPC.

 Шаг 07:

Миссия 7

Сопоставьте этапы настройки gRPC сервиса с их описаниями.

Подсказка: Правильный порядок помогает избежать ошибок при запуске.

Ответ:

- Определение сервиса в .proto
- Компиляция protobuf в код
- Реализация серверной логики
- Запуск сервера и ожидание запросов

1. Создание и запуск процесса, который слушает порты и обрабатывает запросы.
2. Определение методов и сообщений сервиса в protobuf.
3. Перевод .proto файлов в код на выбранном языке.
4. Написание функций, которые реализуют логику методов сервиса.

- **Определение сервиса в .proto** — Определение методов и сообщений сервиса в protobuf.
- **Компиляция protobuf в код** — Перевод .proto файлов в код на выбранном языке.
- **Реализация серверной логики** — Написание функций, которые реализуют логику методов сервиса.
- **Запуск сервера и ожидание запросов** — Создание и запуск процесса, который слушает порты и обрабатывает запросы.

 Шаг 08:

Миссия 8

Расположите шаги создания эффективного gRPC сервиса в правильном порядке:

1. Определение API в .proto файлах
2. Тестирование и отладка сервиса
3. Реализация серверной логики
4. Компиляция protobuf и генерация клиентского и серверного кода

Подсказка: Подумайте о логичном цикле разработки, начиная с планирования и заканчивая проверкой.

Ответ:

- Определение API в .proto файлах
- Компиляция protobuf и генерация клиентского и серверного кода
- Реализация серверной логики
- Тестирование и отладка сервиса

 Шаг 09:**Миссия 9**

Отладка производительности: Расположите этапы профилирования gRPC сервиса в правильном порядке:

1. Анализ результатов и выявление узких мест
2. Настройка инструментов мониторинга и логирования
3. Сбор данных о нагрузке и времени ответа
4. Оптимизация кода и архитектуры на основе данных

Подсказка: Процесс профилирования начинается с подготовки и заканчивается улучшениями.

Ответ:

- Настройка инструментов мониторинга и логирования
- Сбор данных о нагрузке и времени ответа
- Анализ результатов и выявление узких мест
- Оптимизация кода и архитектуры на основе данных

 Шаг 10:**Миссия 10**

Оптимизация безопасности gRPC сервиса: Расположите шаги настройки TLS в правильном порядке:

1. Генерация сертификатов (серверных и клиентских)
2. Настройка gRPC сервера для использования сертификатов
3. Конфигурация клиента для проверки сертификатов сервера
4. Тестирование защищенного соединения

Подсказка: Безопасность — это последовательность от создания ключей к проверке и тестированию.

Ответ:

- Генерация сертификатов (серверных и клиентских)
- Настройка gRPC сервера для использования сертификатов
- Конфигурация клиента для проверки сертификатов сервера
- Тестирование защищенного соединения

Модуль 13 - ♦ Финальный проект (Capstone)

 **Урок 1 - Сервис пользователей**

 **Урок 2 - Сервис чатов**

 **Урок 3 - Сервис статистики**

 **Урок 4 - gRPC-gateway + REST + gRPC-web**

 **Урок 5 - Авторизация через JWT + mTLS**

 **Урок 6 - Метрики, дашборды и алерты**

 **Урок 7 - CI/CD pipeline**

 **Урок 8 - Нагрузочное тестирование**

 **Урок 9 - Викторина 1**

Шаг 01:

Миссия 1

Команда начала проектирование gRPC-сервиса. Какой протокол передачи данных используется по умолчанию в gRPC?

Подсказка: Это бинарный протокол, разработанный Google для высокой производительности.

Ответ:

- **Protocol Buffers**
- JSON
- XML
- Thrift

Шаг 02:

Миссия 2

Команда обсуждает виды взаимодействия в gRPC. Какой тип вызова позволяет обмениваться сообщениями одновременно между клиентом и сервером?

Подсказка: Представьте диалог, где обе стороны говорят одновременно, без ожидания ответа.

Ответ:

- **Двухнаправленный стриминг (Bidirectional Streaming)**
- Однонаправленный стриминг (Client Streaming)
- Одноразовый вызов (Unary)
- Серверный стриминг (Server Streaming)

Шаг 03:

Миссия 3

Команда хочет обработать ошибку, когда клиент не авторизован. Какой код статуса gRPC лучше всего использовать?

Подсказка: Подумайте о статусе, который соответствует ошибке доступа без аутентификации.

Ответ:

- **UNAUTHENTICATED**
- PERMISSION_DENIED
- NOT_FOUND
- INVALID_ARGUMENT

Шаг 04:

Миссия 4

Разработчики хотят разбить большой protobuf-файл на несколько меньших. Как правильно организовать связи между файлами?

Подсказка: Подумайте, как в обычном коде подключаются модули или библиотеки.

Ответ:

- **Использовать директиву `import` с указанием относительного пути к файлу**
- Объединять все определения в одном файле без импортов
- Импортировать скомпилированные бинарные файлы

Шаг 05:

Миссия 5

Сопоставьте типы gRPC вызовов с их описаниями.

Подсказка: Обратите внимание, кто и как отправляет сообщения, и сколько раз.

Ответ:

- Unary
 - Server Streaming
 - Client Streaming
 - Bidirectional Streaming
1. Клиент отправляет одно сообщение, сервер отвечает одним.
 2. Клиент отправляет одно сообщение, сервер отправляет поток сообщений.
 3. Клиент отправляет поток сообщений, сервер отвечает одним сообщением.
 4. Клиент и сервер обмениваются потоками сообщений одновременно.
- **Unary** — Клиент отправляет одно сообщение, сервер отвечает одним.
 - **Server Streaming** — Клиент отправляет одно сообщение, сервер отправляет поток сообщений.
 - **Client Streaming** — Клиент отправляет поток сообщений, сервер отвечает одним сообщением.
 - **Bidirectional Streaming** — Клиент и сервер обмениваются потоками сообщений одновременно.

Шаг 06:

Миссия 6

Сопоставьте основные компоненты gRPC с их описаниями.

Подсказка: Помните, что некоторые компоненты обеспечивают связь, а другие — логику или контроль.

Ответ:

- Channel
- Stub

- **Interceptor**
- **Deadline**

1. Объект для управления соединением с сервером.
2. Автоматически сгенерированный клиент для вызова методов сервиса.
3. Механизм перехвата вызовов для логирования или аутентификации.
4. Временное ограничение на выполнение вызова.

- **Channel** — Объект для управления соединением с сервером.
- **Stub** — Автоматически сгенерированный клиент для вызова методов сервиса.
- **Interceptor** — Механизм перехвата вызовов для логирования или аутентификации.
- **Deadline** — Временное ограничение на выполнение вызова.

Шаг 07:

Миссия 7

Сопоставьте этапы настройки gRPC сервиса с правильным порядком.

Подсказка: Правильный порядок позволяет избежать ошибок при запуске сервиса.

Ответ:

- Определение сервиса в .proto
 - Компиляция protobuf в код
 - Реализация серверной логики
 - Запуск сервера и ожидание запросов
1. Определение методов и сообщений сервиса в protobuf.
 2. Перевод .proto файлов в код на выбранном языке.
 3. Написание функций с логикой методов сервиса.
 4. Создание и запуск процесса, который слушает порты и обрабатывает запросы.
- **Определение сервиса в .proto** — Определение методов и сообщений сервиса в protobuf.
 - **Компиляция protobuf в код** — Перевод .proto файлов в код на выбранном языке.
 - **Реализация серверной логики** — Написание функций с логикой методов сервиса.
 - **Запуск сервера и ожидание запросов** — Создание и запуск процесса, который слушает порты и обрабатывает запросы.

Шаг 08:

Миссия 8

Расположите этапы разработки gRPC-сервиса в правильном порядке:

1. Определение API в .proto файлах
2. Компиляция protobuf и генерация клиентского и серверного кода
3. Реализация серверной логики
4. Тестирование и отладка сервиса

Подсказка: Логичный процесс начинается с проектирования и заканчивается проверкой.

Ответ:

- Определение API в .proto файлах
- Компиляция protobuf и генерация клиентского и серверного кода
- Реализация серверной логики
- Тестирование и отладка сервиса

Урок 10 - Викторина 2

Шаг 01:

Миссия 1

Команда разрабатывает gRPC-сервис для обработки запросов от мобильных клиентов. Какой протокол используется gRPC по умолчанию?

Подсказка: Подумайте, что обеспечивает быструю и эффективную коммуникацию между сервисами.

Ответ:

- **HTTP/2**
- HTTP/1.1
- FTP
- WebSocket

Шаг 02:

Миссия 2

Ваша команда хочет улучшить производительность gRPC-приложения. Что стоит оптимизировать в первую очередь?

Подсказка: Рассмотрите минимизацию сетевых задержек и скорость обработки сообщений.

Ответ:

- **Размер и сериализацию сообщений**
- Цвет интерфейса пользователя
- Использование устаревших протоколов
- Увеличение времени ожидания в таймерах

Шаг 03:

Миссия 3

При разработке нового RPC метода в gRPC, что важно указать в proto-файле?

Подсказка: Вспомните, как описываются структура данных и методы в gRPC.

Ответ:

- **Определение service и gRPC методов с типами сообщений**
- Только описание UI элементов
- Объявление переменных окружения
- Только текстовые комментарии

Шаг 04:

Миссия 4

Сопоставьте термины gRPC с их правильными определениями:

Подсказка: Вспомните ключевые компоненты архитектуры gRPC.

Ответ:

- Proto-файл
- Stub
- Interceptor

- Channel

1. Клиентская сторона, которая вызывает удалённые методы
2. Файл с определениями сервисов и сообщений
3. Механизм перехвата и обработки вызовов
4. Абстракция канала связи между клиентом и сервером

- **Proto-файл** — Файл с определениями сервисов и сообщений
- **Stub** — Клиентская сторона, которая вызывает удалённые методы
- **Interceptor** — Механизм перехвата и обработки вызовов
- **Channel** — Абстракция канала связи между клиентом и сервером

🔗 Шаг 05:

Миссия 5

Свяжите типы gRPC-вызовов с их особенностями:

Подсказка: Подумайте о потоке данных между клиентом и сервером.

Ответ:

- Unary
 - Server Streaming
 - Client Streaming
 - Bidirectional Streaming
1. Один запрос - один ответ
 2. Клиент отправляет поток, сервер отвечает одним сообщением
 3. Сервер отправляет поток ответов на один запрос клиента
 4. Двусторонний обмен потоками сообщений
- **Unary** — Один запрос - один ответ
 - **Server Streaming** — Сервер отправляет поток ответов на один запрос клиента
 - **Client Streaming** — Клиент отправляет поток, сервер отвечает одним сообщением
 - **Bidirectional Streaming** — Двусторонний обмен потоками сообщений

🔗 Шаг 06:

Миссия 6

Соотнесите задачи в разработке gRPC с правильными инструментами:

Подсказка: Инструменты помогают оптимизировать разработку и отладку.

Ответ:

- Проверка схемы proto
 - Трассировка вызовов
 - Мониторинг производительности
 - Генерация кода
1. protoc
 2. Jaeger или Zipkin
 3. Prometheus
 4. protoc-gen-* плагины
- **Проверка схемы proto** — protoc
 - **Трассировка вызовов** — Jaeger или Zipkin
 - **Мониторинг производительности** — Prometheus

- **Генерация кода** — protoc-gen-* плагины

Шаг 07:

Миссия 7

Отсортируйте этапы отладки gRPC-сервиса в правильном порядке:

Подсказка: Начните с воспроизведения проблемы, затем переходите к поиску причины.

1. Запуск сервиса в режиме отладки
2. Сбор логов и метрик
3. Воспроизведение ошибки с клиента
4. Анализ и исправление кода

Ответ:

- Воспроизведение ошибки с клиента
- Запуск сервиса в режиме отладки
- Сбор логов и метрик
- Анализ и исправление кода

Шаг 08:

Миссия 8

Расставьте шаги для создания нового gRPC метода на сервере:

Подсказка: Сначала опишите интерфейс, затем реализуйте и протестируйте.

1. Написание реализации метода в серверном коде
2. Добавление определения метода в proto-файл
3. Генерация кода из proto-файла
4. Тестирование нового метода

Ответ:

- Добавление определения метода в proto-файл
- Генерация кода из proto-файла
- Написание реализации метода в серверном коде
- Тестирование нового метода

Шаг 09:

Миссия 9

Упорядочьте действия для оптимизации gRPC-сервиса:

Подсказка: Начинайте с мониторинга и анализа, затем применяйте улучшения.

1. Анализ узких мест с помощью метрик
2. Реализация кэширования ответов
3. Мониторинг использования ресурсов
4. Тестирование изменений под нагрузкой

Ответ:

- Мониторинг использования ресурсов
- Анализ узких мест с помощью метрик
- Реализация кэширования ответов
- Тестирование изменений под нагрузкой

Шаг 10:

Миссия 10

Расположите этапы CI/CD для gRPC-приложения:

Подсказка: Автоматизация требует последовательного подхода к сборке, тестированию и деплою.

1. Развертывание на тестовом окружении
2. Сборка артефактов
3. Автоматическое тестирование
4. Запуск сборки по коммиту

Ответ:

- Запуск сборки по коммиту
- Сборка артефактов
- Автоматическое тестирование
- Развертывание на тестовом окружении